

Dental Management System

Full Code Walkthrough — Every File Explained

A file-by-file guide to the backend (Node.js + Express + MongoDB) and the frontend (React + Tailwind CSS), written so you can explain the whole project to a friend, file by file, without reading the entire codebase yourself.

How to Use This Document

This is a MERN app: **M**ongoDB (database), **E**xpress (backend web framework running on Node.js), **R**ect (frontend UI library), **N**ode.js (JavaScript runtime for the backend). The project has two independent folders that talk to each other over HTTP requests:

- **backend/** — a REST API server with no visual interface. It only accepts requests like "give me all patients" or "create an appointment", talks to MongoDB, and sends back JSON data.
- **frontend/** — the React website a user actually sees and clicks around in. It calls the backend's API with Axios (an HTTP request library) whenever it needs data, and renders that data as HTML.

For every file below there are two parts: **"What this file does"** in plain English, and **"The code"** — the exact code written in that file. Read the explanation first, then match it against the code underneath. That combination is the fastest way to explain any single file to someone else without memorizing it.

Files are grouped in the order you'd naturally read them: Backend first (server entry point, then database models, then middleware, then API routes), then Frontend (entry point, then shared contexts, then reusable components, then the actual pages).

PART 1 — BACKEND (Node.js + Express + MongoDB)

The backend lives in the **backend/** folder. It's a Node.js app built with the Express framework. Its whole job is: receive an HTTP request → check if the user is logged in → read/write data in MongoDB → send back a JSON response. It never renders any HTML — that's the frontend's job.

backend/server.js

WHAT THIS FILE DOES

This is the starting point of the entire backend — the file you run with `node server.js`. It creates the Express application, turns on two plugins (CORS so the React app running on a different port is allowed to call this API, and a JSON body parser so `req.body` works), then tells Express which route file handles which URL prefix (e.g. anything starting with `/api/patients` goes to `patients.js`). Finally it connects to MongoDB using Mongoose and only starts listening for requests once that's done.

- `app.use(cors())` and `app.use(express.json())` are 'middleware' — code that runs on every single request before it reaches a route.
- Each `app.use('/api/xxx', xxxRoutes)` line mounts one route file under a URL prefix — this is how the 12 separate route files are wired into one server.
- The `mongoose.connect(...)` call includes a 5-second timeout so the server fails fast and prints a clear error instead of hanging forever if MongoDB isn't reachable.
- Even if MongoDB fails to connect, the server still starts (with a warning) so you can at least see the API is alive — useful for debugging.

THE CODE

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const authRoutes = require("./routes/auth");
const patientRoutes = require("./routes/patients");
const dentistRoutes = require("./routes/dentists");
const appointmentRoutes = require("./routes/appointments");
const invoiceRoutes = require("./routes/invoices");
const dashboardRoutes = require("./routes/dashboard");
const userRoutes = require("./routes/users");
const serviceRoutes = require("./routes/services");
const notificationRoutes = require("./routes/notifications");
const recordRoutes = require("./routes/records");
const uploadRoutes = require("./routes/upload");
const patientAuthRoutes = require("./routes/patientAuth");
const portalRoutes = require("./routes/portal");
const { startReminderCron } = require("./config/cron");

const app = express();
app.use(cors());
app.use(express.json());
app.use("/uploads", express.static(require("path").join(__dirname, "uploads")));

app.use("/api/auth", authRoutes);
app.use("/api/patients", patientRoutes);
app.use("/api/dentists", dentistRoutes);
app.use("/api/appointments", appointmentRoutes);
app.use("/api/invoices", invoiceRoutes);
app.use("/api/dashboard", dashboardRoutes);
app.use("/api/users", userRoutes);
app.use("/api/services", serviceRoutes);
app.use("/api/notifications", notificationRoutes);
app.use("/api/records", recordRoutes);
app.use("/api/upload", uploadRoutes);
app.use("/api/patient-auth", patientAuthRoutes);
app.use("/api/portal", portalRoutes);

app.get("/", (req, res) => res.send("Dental Management API running"));
```

```

const PORT = process.env.PORT || 5000;

mongoose
  .connect(process.env.MONGO_URI || "mongodb://127.0.0.1:27017/dental_management", {
    serverSelectionTimeoutMS: 5000,
  })
  .then(() => {
    console.log("MongoDB connected");
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      startReminderCron();
    });
  })
  .catch((err) => {
    console.error("MongoDB connection error:", err.message);
    // still start server so API structure can be inspected/tested
    app.listen(PORT, () => console.log(`Server running on port ${PORT} (DB not connected)`));
  });

module.exports = app;

```

Database Models (backend/models/)

A 'model' is a JavaScript blueprint for what one document (record) in a MongoDB collection looks like — its fields, their types, and any rules (like 'required' or 'must be one of these values'). Mongoose turns each model into an easy-to-use object: e.g. `Patient.find()` fetches all patients, `Patient.create(data)` inserts a new one.

backend/models/User.js

WHAT THIS FILE DOES

Defines a staff/admin account: name, email, a hashed password, a role ("admin" or "staff"), and whether the account is active. This is the only model with custom logic attached to it, because passwords need special handling — you must never store a plain-text password in the database.

- `userSchema.pre('save', ...)` runs automatically right before any user document is saved. If the password field was changed, it replaces it with a bcrypt hash (a one-way scrambled version) using `bcrypt.hash()`.
- `comparePassword()` is used at login time: it takes the password the person just typed and asks bcrypt to check it against the stored hash, without ever un-hashing anything.

THE CODE

```

const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    role: { type: String, enum: ["admin", "staff"], default: "staff" },
    isActive: { type: Boolean, default: true },
    phone: String,
    avatar: String,
  },
  { timestamps: true }
);

userSchema.methods.comparePassword = function (candidate) {
  return bcrypt.compare(candidate, this.password);
};

userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

module.exports = mongoose.model("User", userSchema);

```

backend/models/Patient.js

WHAT THIS FILE DOES

Defines what information is stored about a single dental patient: name, age, gender, phone, email, address, blood group, medical history/allergies notes, and an optional photo path. `timestamps: true` automatically adds `createdAt/updatedAt` fields.

THE CODE

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const patientSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    age: Number,
    gender: { type: String, enum: ["Male", "Female", "Other"] },
    phone: { type: String, required: true },
    email: { type: String, unique: true, sparse: true }, // sparse: many old records may have no email
    address: String,
    medicalHistory: String,
    bloodGroup: String,
    photo: String,

    // --- Patient portal login (added for self-service portal) ---
    password: { type: String, select: false }, // only set once the patient registers for portal access
    hasPortalAccess: { type: Boolean, default: false },
    emailVerified: { type: Boolean, default: false },
  },
  { timestamps: true }
);

patientSchema.methods.comparePassword = function (candidate) {
  return bcrypt.compare(candidate, this.password);
};

patientSchema.pre("save", async function (next) {
  if (!this.isModified("password") || !this.password) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

module.exports = mongoose.model("Patient", patientSchema);
```

backend/models/Dentist.js

WHAT THIS FILE DOES

Defines a dentist's profile: name, specialization (e.g. "Orthodontist"), phone, email, and years of experience. Kept intentionally simple since dentists are managed like a small staff directory.

THE CODE

```
const mongoose = require("mongoose");

const dentistSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    specialization: String,
    phone: String,
    email: String,
    experienceYears: Number,
  },
  { timestamps: true }
);

module.exports = mongoose.model("Dentist", dentistSchema);
```

backend/models/Appointment.js

WHAT THIS FILE DOES

Defines one booked visit. Instead of storing the patient's and dentist's full details again, it stores their MongoDB IDs as references (ref: "Patient" / ref: "Dentist"). This is how relationships work in MongoDB — later, routes use .populate() to automatically swap that ID for the real patient/dentist object when reading the data back.

- The status field is restricted to exactly three values with enum: Scheduled, Completed, or Cancelled — MongoDB will reject anything else.

THE CODE

```
const mongoose = require("mongoose");

const appointmentSchema = new mongoose.Schema(
  {
    patient: { type: mongoose.Schema.Types.ObjectId, ref: "Patient", required: true },
    dentist: { type: mongoose.Schema.Types.ObjectId, ref: "Dentist", required: true },
    date: { type: Date, required: true },
    time: { type: String, required: true },
    reason: String,
    status: {
      type: String,
      enum: ["Scheduled", "Completed", "Cancelled"],
      default: "Scheduled",
    },
    notes: String,
    reminderSent: { type: Boolean, default: false },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Appointment", appointmentSchema);
```

backend/models/Invoice.js

WHAT THIS FILE DOES

Defines a bill for a patient. Notice invoiceItemSchema is a small schema used *inside* the main schema — each invoice can have multiple treatment line-items (e.g. "Cleaning — ■500", "X-ray — ■300"), each with its own cost.

- totalAmount is the full bill, paidAmount is how much has actually been paid so far — the difference is the outstanding balance shown in the UI.
- status (Unpaid / Partially Paid / Paid) is calculated and sent in by the frontend when the invoice is created, based on how much was paid versus the total.

THE CODE

```
const mongoose = require("mongoose");

const invoiceItemSchema = new mongoose.Schema({
  treatment: { type: String, required: true },
  cost: { type: Number, required: true },
});

const invoiceSchema = new mongoose.Schema(
  {
    patient: { type: mongoose.Schema.Types.ObjectId, ref: "Patient", required: true },
    appointment: { type: mongoose.Schema.Types.ObjectId, ref: "Appointment" },
    items: [invoiceItemSchema],
    totalAmount: { type: Number, required: true },
    paidAmount: { type: Number, default: 0 },
    status: {
      type: String,
      enum: ["Unpaid", "Partially Paid", "Paid"],
      default: "Unpaid",
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Invoice", invoiceSchema);
```

backend/models/Service.js

WHAT THIS FILE DOES

Defines one row in the clinic's treatment price list (used on the Services page): a name, a category for grouping (like "Preventive" or "Surgery"), a price, and an estimated duration in minutes. This exists so that when creating an invoice, staff can pick a known treatment and have its price fill in automatically instead of typing the price by hand every time.

THE CODE

```
const mongoose = require("mongoose");

const serviceSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    category: { type: String, default: "General" },
    price: { type: Number, required: true },
    durationMinutes: { type: Number, default: 30 },
    description: String,
  },
  { timestamps: true }
);

module.exports = mongoose.model("Service", serviceSchema);
```

backend/models/MedicalRecord.js

WHAT THIS FILE DOES

Defines one clinical note for a patient visit: diagnosis, treatment actually done, prescription, free-text notes, which dentist saw the patient, and an attachments array for uploaded files (like X-ray images or scanned documents). This is what powers the "Medical Records" tab on a patient's profile page.

THE CODE

```
const mongoose = require("mongoose");

const medicalRecordSchema = new mongoose.Schema(
  {
    patient: { type: mongoose.Schema.Types.ObjectId, ref: "Patient", required: true },
    dentist: { type: mongoose.Schema.Types.ObjectId, ref: "Dentist" },
    appointment: { type: mongoose.Schema.Types.ObjectId, ref: "Appointment" },
    visitDate: { type: Date, default: Date.now },
    diagnosis: String,
    treatmentDone: String,
    prescription: String,
    notes: String,
    attachments: [{ filename: String, url: String }],
  },
  { timestamps: true }
);

module.exports = mongoose.model("MedicalRecord", medicalRecordSchema);
```

backend/models/Notification.js

WHAT THIS FILE DOES

A very small model used to power the notification bell in the top corner of the app. Every notification just has a message, a type (appointment / payment / system, which decides which emoji icon is shown), and a read flag so the UI knows whether to count it as "unread".

THE CODE

```
const mongoose = require("mongoose");

const notificationSchema = new mongoose.Schema(
  {
    message: { type: String, required: true },
    type: { type: String, enum: ["appointment", "payment", "system"], default: "system" },
    read: { type: Boolean, default: false },
  },
  { timestamps: true }
);
```

```
);  
module.exports = mongoose.model("Notification", notificationSchema);
```

Middleware (backend/middleware/)

Middleware is code that sits *in front of* a route and runs first. If it calls `next()`, the request is allowed to continue to the actual route handler. If it doesn't, it sends back an error response and the route code never runs at all.

backend/middleware/auth.js

WHAT THIS FILE DOES

This is the security gate for almost every API route. It exports two functions.

- `auth` (the default export) reads the `Authorization: Bearer <token>` header sent by the frontend, verifies the JWT token's signature with `jwt.verify()`, and if it's valid, attaches the decoded user info (`id`, `role`, `name`) onto `req.user` so every later route can see who's making the request. If the token is missing or invalid, it immediately responds with 401 Unauthorized and stops.
- `adminOnly` is a second, extra check used only on routes that should be restricted to admins (like deleting a staff member or editing the price list). It assumes `auth` already ran and simply checks `req.user.role === "admin"`, responding 403 Forbidden if not.

THE CODE

```
const jwt = require("jsonwebtoken");  
  
function auth(req, res, next) {  
  const header = req.headers.authorization;  
  if (!header || !header.startsWith("Bearer ")) {  
    return res.status(401).json({ message: "No token provided" });  
  }  
  const token = header.split(" ")[1];  
  try {  
    const decoded = jwt.verify(token, process.env.JWT_SECRET || "change_this_secret_key");  
    req.user = decoded;  
    next();  
  } catch (err) {  
    return res.status(401).json({ message: "Invalid or expired token" });  
  }  
}  
  
function adminOnly(req, res, next) {  
  if (!req.user || req.user.role !== "admin") {  
    return res.status(403).json({ message: "Admin access required" });  
  }  
  next();  
}  
  
module.exports = auth;  
module.exports.adminOnly = adminOnly;
```

Configuration (backend/config/)

backend/config/upload.js

WHAT THIS FILE DOES

Configures Multer, the library that handles file uploads (used for X-ray/document attachments on medical records). Uploaded files are saved straight to disk in an `uploads/` folder rather than into MongoDB, which keeps the database small and fast.

- `diskStorage` decides where files go and what to name them — each filename gets a random-looking prefix (`Date.now() + random number`) so two people uploading a file called `photo.jpg` at the same time never overwrite each other.

- `limits: { fileSize: 5MB }` stops someone from uploading a huge file that could slow down or crash the server.
- `fileFilter` only allows image types and PDFs — anything else (like a `.exe` file) is rejected before it's even saved.

THE CODE

```
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const uploadDir = path.join(__dirname, "..", "uploads");
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const unique = Date.now() + "-" + Math.round(Math.random() * 1e9);
    cb(null, unique + path.extname(file.originalname));
  },
});

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    const allowed = /jpeg|jpg|png|webp|pdf/;
    const ok = allowed.test(path.extname(file.originalname).toLowerCase());
    cb(ok ? null : new Error("Only images and PDFs are allowed"), ok);
  },
});

module.exports = upload;
```

API Routes (backend/routes/)

Each file in this folder defines one group of related endpoints, all mounted under a URL prefix in `server.js` (e.g. everything in `patients.js` lives under `/api/patients/...`). Almost every route file starts with `router.use(auth)`, which forces every route inside that file to require a valid login — the only exception is `auth.js` itself, since you obviously can't be logged in yet when you're trying to log in.

backend/routes/auth.js

WHAT THIS FILE DOES

Handles everything about logging in and account identity: registering a new staff account, logging in an existing one, fetching "my own" profile, and changing your own password. This is the one route file that doesn't require a token for its first two routes, since you need to log in *before* you have a token.

- `genToken()` is a small helper that packs the user's id/role/name into a signed JWT that expires after 7 days — this token is what the frontend stores and sends on every future request.
- `POST /register` checks the email isn't already taken, creates the user (which triggers the password-hashing hook in the `User` model), and immediately returns a login token so the person doesn't have to log in separately right after signing up.
- `POST /login` looks the user up by email, then uses `comparePassword()` from the `User` model to check the password before issuing a token.

THE CODE

```
const router = require("express").Router();
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const genToken = (user) => {
  jwt.sign({ id: user._id, role: user.role, name: user.name }, process.env.JWT_SECRET || "change_this_secret_key", {
    expiresIn: "7d",
  });
};

// Register (first admin / staff signup)
router.post("/register", async (req, res) => {
  try {
```

```

    const { name, email, password, role } = req.body;
    const exists = await User.findOne({ email });
    if (exists) return res.status(400).json({ message: "Email already registered" });
    const user = await User.create({ name, email, password, role });
    res.status(201).json({ token: genToken(user), user: { id: user._id, name: user.name, role: user.role } });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// Login
router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user) return res.status(400).json({ message: "Invalid credentials" });
    const match = await user.comparePassword(password);
    if (!match) return res.status(400).json({ message: "Invalid credentials" });
    res.json({ token: genToken(user), user: { id: user._id, name: user.name, role: user.role } });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// Get current logged-in user's profile
router.get("/me", require("../middleware/auth"), async (req, res) => {
  const user = await User.findById(req.user.id).select("-password");
  res.json(user);
});

// Update current user's name
router.put("/me", require("../middleware/auth"), async (req, res) => {
  const { name } = req.body;
  const user = await User.findByIdAndUpdate(req.user.id, { name }, { new: true }).select("-password");
  res.json(user);
});

// Change current user's password
router.put("/me/password", require("../middleware/auth"), async (req, res) => {
  try {
    const { currentPassword, newPassword } = req.body;
    const user = await User.findById(req.user.id);
    const match = await user.comparePassword(currentPassword);
    if (!match) return res.status(400).json({ message: "Current password is incorrect" });
    user.password = newPassword;
    await user.save();
    res.json({ message: "Password updated" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

module.exports = router;

```

backend/routes/patients.js

WHAT THIS FILE DOES

The busiest route file in the project. It handles every operation on patient records, plus some extra reporting features.

- GET / supports ?search=&gender=&page=&limit= query parameters: it builds a MongoDB filter with a case-insensitive regex search across name/phone/email, an optional gender filter, then uses .skip().limit() for pagination, and returns the total count too so the frontend can show "Page 2 of 5".
- GET /export/csv pulls every patient, converts the array of objects into CSV text using the json2csv library, and sends it back with headers that make the browser download it as a file instead of displaying it.
- GET /:id/full is a convenience endpoint for the patient profile page: instead of the frontend making 3 separate requests, this one route fetches the patient plus all their appointments and invoices in parallel (Promise.all) and returns them together.
- Notice the more specific routes (/export/csv, /:id/full) are defined *before* the generic /:id route — Express checks routes top to bottom, so if /:id came first it would wrongly try to treat "export" as an ID.

THE CODE

```
const router = require("express").Router();
```

```

const { Parser } = require("json2csv");
const Patient = require("../models/Patient");
const Appointment = require("../models/Appointment");
const Invoice = require("../models/Invoice");
const auth = require("../middleware/auth");

router.use(auth);

// Supports ?search=&gender=&page=&limit=
router.get("/", async (req, res) => {
  const { search = "", gender = "", page = 1, limit = 10 } = req.query;
  const query = {};
  if (search) {
    query.$or = [
      { name: new RegExp(search, "i") },
      { phone: new RegExp(search, "i") },
      { email: new RegExp(search, "i") },
    ];
  }
  if (gender) query.gender = gender;

  const skip = (Number(page) - 1) * Number(limit);
  const [patients, total] = await Promise.all([
    Patient.find(query).sort({ createdAt: -1 }).skip(skip).limit(Number(limit)),
    Patient.countDocuments(query),
  ]);
  res.json({ patients, total, page: Number(page), pages: Math.ceil(total / limit) || 1 });
});

// Export all patients as CSV
router.get("/export/csv", async (req, res) => {
  try {
    const patients = await Patient.find().sort({ name: 1 }).lean();
    const fields = ["name", "age", "gender", "phone", "email", "address", "bloodGroup", "medicalHistory"];
    const parser = new Parser({ fields });
    const csv = parser.parse(patients);
    res.header("Content-Type", "text/csv");
    res.attachment("patients.csv");
    res.send(csv);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// Full patient profile: patient + appointments + invoices + medical records
router.get("/:id/full", async (req, res) => {
  const patient = await Patient.findById(req.params.id);
  if (!patient) return res.status(404).json({ message: "Patient not found" });

  const [appointments, invoices] = await Promise.all([
    Appointment.find({ patient: patient._id }).populate("dentist", "name specialization").sort({ date: -1 }),
    Invoice.find({ patient: patient._id }).sort({ createdAt: -1 }),
  ]);

  res.json({ patient, appointments, invoices });
});

router.get("/:id", async (req, res) => {
  const patient = await Patient.findById(req.params.id);
  if (!patient) return res.status(404).json({ message: "Patient not found" });
  res.json(patient);
});

router.post("/", async (req, res) => {
  try {
    const patient = await Patient.create(req.body);
    res.status(201).json(patient);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

router.put("/:id", async (req, res) => {
  const patient = await Patient.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(patient);
});

router.delete("/:id", async (req, res) => {
  await Patient.findByIdAndDelete(req.params.id);
  res.json({ message: "Patient deleted" });
});

module.exports = router;

```

backend/routes/dentists.js

WHAT THIS FILE DOES

The simplest CRUD (Create, Read, Update, Delete) route file in the project — a good one to study first if you're new to Express. Four routes: list all dentists, add one, update one by id, delete one by id. No filtering or pagination since a clinic typically only has a handful of dentists.

THE CODE

```
const router = require("express").Router();
const Dentist = require("../models/Dentist");
const auth = require("../middleware/auth");

router.use(auth);

router.get("/", async (req, res) => {
  const dentists = await Dentist.find().sort({ createdAt: -1 });
  res.json(dentists);
});

router.post("/", async (req, res) => {
  try {
    const dentist = await Dentist.create(req.body);
    res.status(201).json(dentist);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

router.put("/:id", async (req, res) => {
  const dentist = await Dentist.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(dentist);
});

router.delete("/:id", async (req, res) => {
  await Dentist.findByIdAndDelete(req.params.id);
  res.json({ message: "Dentist deleted" });
});

module.exports = router;
```

backend/routes/appointments.js

WHAT THIS FILE DOES

Handles booking and managing appointments, and doubles as the data source for the dashboard's "today's appointments" widget and the notification bell.

- GET / accepts optional status, date, and dentist filters, and always calls .populate() twice so the response includes the actual patient name/phone and dentist name/specialization instead of just their raw MongoDB IDs.
- GET /today/list builds a date range from midnight to 11:59pm today and returns only Scheduled appointments in that window — this is what the Dashboard page shows under "Today's Appointments".
- POST / creates the appointment, then (without making the user wait for it) fires off Notification.create(...) in the background so a bell notification appears for the new booking. The .catch(() => {}) on that call means even if the notification fails to save, the appointment booking itself still succeeds.

THE CODE

```
const router = require("express").Router();
const Appointment = require("../models/Appointment");
const Notification = require("../models/Notification");
const auth = require("../middleware/auth");
const { sendMail } = require("../config/mailer");
const { appointmentConfirmationEmail } = require("../utils/emailTemplates");

router.use(auth);

// Supports ?status=&date=&dentist=
router.get("/", async (req, res) => {
  const { status, date, dentist } = req.query;
  const query = {};
```

```

if (status) query.status = status;
if (dentist) query.dentist = dentist;
if (date) {
  const start = new Date(date);
  const end = new Date(date);
  end.setDate(end.getDate() + 1);
  query.date = { $gte: start, $lt: end };
}
const appointments = await Appointment.find(query)
  .populate("patient", "name phone")
  .populate("dentist", "name specialization")
  .sort({ date: 1 });
res.json(appointments);
});

router.get("/today/list", async (req, res) => {
  const start = new Date();
  start.setHours(0, 0, 0, 0);
  const end = new Date();
  end.setHours(23, 59, 59, 999);
  const appointments = await Appointment.find({ date: { $gte: start, $lte: end }, status: "Scheduled" })
    .populate("patient", "name phone")
    .populate("dentist", "name")
    .sort({ time: 1 });
  res.json(appointments);
});

router.post("/", async (req, res) => {
  try {
    const appointment = await Appointment.create(req.body);
    const populated = await appointment.populate([{ path: "patient", select: "name email" }, { path: "dentist", select: "name" }]);
    Notification.create({
      message: `New appointment: ${populated.patient?.name || "Patient"} with Dr. ${populated.dentist?.name || ""} on ${new Date(appointment.date).toLocaleDateString()}`,
      type: "appointment",
    }).catch(() => {});

    // Step 8: Email confirmation to the patient, if we have an email on file.
    if (populated.patient?.email) {
      sendMail({
        to: populated.patient.email,
        subject: "Your DentalCare Appointment is Confirmed",
        html: appointmentConfirmationEmail({
          patientName: populated.patient.name,
          dentistName: populated.dentist?.name || "",
          date: new Date(appointment.date).toLocaleDateString(),
          time: appointment.time,
          reason: appointment.reason,
        }),
      }).catch((err) => console.error("[appointments] confirmation email failed:", err.message));
    }

    res.status(201).json(appointment);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

router.put("/:id", async (req, res) => {
  const appointment = await Appointment.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(appointment);
});

router.delete("/:id", async (req, res) => {
  await Appointment.findByIdAndDelete(req.params.id);
  res.json({ message: "Appointment deleted" });
});

module.exports = router;

```

backend/routes/invoices.js

WHAT THIS FILE DOES

Handles billing. Very similar shape to appointments.js: list, create, update, delete. Creating an invoice also auto-generates a notification ("New invoice created for ... — ) the same way booking an appointment does. Deleting an invoice is restricted to admins only via the `adminOnly` middleware, since deleting billing history is a

sensitive action.

THE CODE

```
const router = require("express").Router();
const Invoice = require("../models/Invoice");
const Notification = require("../models/Notification");
const auth = require("../middleware/auth");
const { adminOnly } = require("../middleware/auth");
const { sendMail } = require("../config/mailer");
const { paymentReceiptEmail } = require("../utils/emailTemplates");
const { generateReceiptPDF } = require("../utils/pdfGenerator");

router.use(auth);

router.get("/", async (req, res) => {
  const invoices = await Invoice.find().populate("patient", "name phone").sort({ createdAt: -1 });
  res.json(invoices);
});

router.post("/", async (req, res) => {
  try {
    const invoice = await Invoice.create(req.body);
    const populated = await invoice.populate("patient", "name email");
    Notification.create({
      message: `New invoice created for ${populated.patient?.name || "patient"} - $${invoice.totalAmount} (${invoice.status})`,
      type: "payment",
    }).catch(() => {});

    // Step 9: Email the payment receipt (with PDF attached) if the patient has an email on file.
    if (populated.patient?.email) {
      generateReceiptPDF(invoice, populated.patient)
        .then((pdfBuffer) => {
          sendMail({
            to: populated.patient.email,
            subject: "Your DentalCare Payment Receipt",
            html: paymentReceiptEmail({
              patientName: populated.patient.name,
              totalAmount: invoice.totalAmount,
              paidAmount: invoice.paidAmount,
              status: invoice.status,
            }),
            attachments: [{ filename: "receipt.pdf", content: pdfBuffer }],
          })
        })
        .catch((err) => console.error("[invoices] receipt email failed:", err.message));
    }

    res.status(201).json(invoice);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

router.put("/:id", async (req, res) => {
  const invoice = await Invoice.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(invoice);
});

router.delete("/:id", adminOnly, async (req, res) => {
  await Invoice.findByIdAndDelete(req.params.id);
  res.json({ message: "Invoice deleted" });
});

router.get("/:id", async (req, res) => {
  const invoice = await Invoice.findById(req.params.id).populate("patient", "name phone address");
  if (!invoice) return res.status(404).json({ message: "Invoice not found" });
  res.json(invoice);
});

module.exports = router;
```

backend/routes/services.js

WHAT THIS FILE DOES

Manages the treatment price catalog seen on the Services page. Anyone logged in can *view* the list (needed so staff can pick treatments while billing), but only admins can add, edit, or delete a service — enforced by putting

adminOnly as a second argument on just the POST/PUT/DELETE routes, while GET has no such restriction.

THE CODE

```
const router = require("express").Router();
const Service = require("../models/Service");
const auth = require("../middleware/auth");
const { adminOnly } = require("../middleware/auth");

router.use(auth);

router.get("/", async (req, res) => {
  const services = await Service.find().sort({ category: 1, name: 1 });
  res.json(services);
});

router.post("/", adminOnly, async (req, res) => {
  try {
    const service = await Service.create(req.body);
    res.status(201).json(service);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

router.put("/:id", adminOnly, async (req, res) => {
  const service = await Service.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(service);
});

router.delete("/:id", adminOnly, async (req, res) => {
  await Service.findByIdAndDelete(req.params.id);
  res.json({ message: "Service deleted" });
});

module.exports = router;
```

backend/routes/users.js

WHAT THIS FILE DOES

Splits into two halves: routes for managing **your own** account, and admin-only routes for managing **everyone else's** account.

- GET/PUT /me and PUT /me/password let the currently logged-in user view/edit their own name, phone, avatar, or change their own password (re-checking their current password first, same pattern as the User model's login check).
- GET /, PUT /:id, and DELETE /:id are all guarded by adminOnly and power the Staff Management page — promoting/demoting roles, activating/deactivating accounts, and removing staff.
- The delete route has a small but important safety check: `if (req.params.id === req.user.id) return res.status(400) . . .` stops an admin from accidentally deleting their own account and getting locked out.

THE CODE

```
const router = require("express").Router();
const User = require("../models/User");
const auth = require("../middleware/auth");
const { adminOnly } = require("../middleware/auth");

router.use(auth);

router.get("/me", async (req, res) => {
  const user = await User.findById(req.user.id).select("-password");
  res.json(user);
});

router.put("/me", async (req, res) => {
  const { name, phone, avatar } = req.body;
  const user = await User.findByIdAndUpdate(req.user.id, { name, phone, avatar }, { new: true }).select("-password");
  res.json(user);
});

router.put("/me/password", async (req, res) => {
  const { currentPassword, newPassword } = req.body;
  const user = await User.findById(req.user.id);
  const match = await user.comparePassword(currentPassword);
```

```

    if (!match) return res.status(400).json({ message: "Current password is incorrect" });
    user.password = newPassword;
    await user.save();
    res.json({ message: "Password updated" });
  });

// Admin-only: manage staff accounts
router.get("/", adminOnly, async (req, res) => {
  const users = await User.find().select("-password").sort({ createdAt: -1 });
  res.json(users);
});

router.put("/:id", adminOnly, async (req, res) => {
  const { role, isActive } = req.body;
  const update = {};
  if (role) update.role = role;
  if (typeof isActive === "boolean") update.isActive = isActive;
  const user = await User.findByIdAndUpdate(req.params.id, update, { new: true }).select("-password");
  res.json(user);
});

router.delete("/:id", adminOnly, async (req, res) => {
  if (req.params.id === req.user.id) {
    return res.status(400).json({ message: "You can't delete your own account" });
  }
  await User.findByIdAndDelete(req.params.id);
  res.json({ message: "User removed" });
});

module.exports = router;

```

backend/routes/records.js

WHAT THIS FILE DOES

Manages medical records (diagnosis, treatment notes, prescriptions, attachments) for the Medical Records tab on a patient's profile. GET /patient/:patientId is the main read route — it fetches every record for one specific patient, most recent visit first, with the dentist's name populated in.

THE CODE

```

const router = require("express").Router();
const MedicalRecord = require("../models/MedicalRecord");
const auth = require("../middleware/auth");

router.use(auth);

// Get all records for a given patient
router.get("/patient/:patientId", async (req, res) => {
  const records = await MedicalRecord.find({ patient: req.params.patientId })
    .populate("dentist", "name specialization")
    .sort({ visitDate: -1 });
  res.json(records);
});

router.post("/", async (req, res) => {
  try {
    const record = await MedicalRecord.create(req.body);
    res.status(201).json(record);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

router.put("/:id", async (req, res) => {
  const record = await MedicalRecord.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(record);
});

router.delete("/:id", async (req, res) => {
  await MedicalRecord.findByIdAndDelete(req.params.id);
  res.json({ message: "Record deleted" });
});

module.exports = router;

```


backend/routes/upload.js

WHAT THIS FILE DOES

A tiny, generic file-upload endpoint. It uses the Multer config from `config/upload.js` as middleware (`upload.single("file")`) to handle one uploaded file per request, then responds with the URL the frontend can use to later display or download that file. This is called by the PatientDetail page right before it saves a new medical record with an attachment.

THE CODE

```
const router = require("express").Router();
const upload = require("../config/upload");
const auth = require("../middleware/auth");

router.use(auth);

// Generic single-file upload endpoint. Returns a URL usable by the frontend.
router.post("/", upload.single("file"), (req, res) => {
  if (!req.file) return res.status(400).json({ message: "No file uploaded" });
  res.json({ url: `/uploads/${req.file.filename}`, filename: req.file.originalname });
});

module.exports = router;
```

backend/routes/notifications.js

WHAT THIS FILE DOES

Powers the bell icon in the sidebar. GET `/` returns the 30 most recent notifications; GET `/unread-count` returns just a number (used to show the little red badge without downloading the full list); PUT `/read-all` marks everything as read in one database update when the user clicks "Mark all read".

THE CODE

```
const router = require("express").Router();
const Notification = require("../models/Notification");
const auth = require("../middleware/auth");

router.use(auth);

router.get("/", async (req, res) => {
  const notifications = await Notification.find().sort({ createdAt: -1 }).limit(30);
  res.json(notifications);
});

router.get("/unread-count", async (req, res) => {
  const count = await Notification.countDocuments({ read: false });
  res.json({ count });
});

router.put("/:id/read", async (req, res) => {
  const n = await Notification.findByIdAndUpdate(req.params.id, { read: true }, { new: true });
  res.json(n);
});

router.put("/read-all", async (req, res) => {
  await Notification.updateMany({ read: false }, { read: true });
  res.json({ message: "All marked as read" });
});

module.exports = router;
```

backend/routes/dashboard.js

WHAT THIS FILE DOES

Calculates all the numbers and chart data shown on the Dashboard page in one single request, so the frontend doesn't need five separate API calls.

- Runs patient count, dentist count, scheduled-appointment count, all invoices, and an appointment-status aggregation all **at the same time** with `Promise.all` instead of one after another, which makes the dashboard load faster.
- revenue and pending are calculated in JavaScript by looping over every invoice and summing `paidAmount` and `totalAmount - paidAmount`.
- The revenue-trend chart data is built by generating the last 6 calendar months, then bucketing every invoice's paid amount into the month it was created in — this is what feeds the line chart on the dashboard.

THE CODE

```
const router = require("express").Router();
const Patient = require("../models/Patient");
const Dentist = require("../models/Dentist");
const Appointment = require("../models/Appointment");
const Invoice = require("../models/Invoice");
const auth = require("../middleware/auth");

router.use(auth);

const MONTHS = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];

router.get("/stats", async (req, res) => {
  const [patients, dentists, upcomingAppointments, invoices, statusAgg] = await Promise.all([
    Patient.countDocuments(),
    Dentist.countDocuments(),
    Appointment.countDocuments({ status: "Scheduled" }),
    Invoice.find(),
    Appointment.aggregate([
      { $group: { _id: "$status", count: { $sum: 1 } } }
    ]
  ]);

  const revenue = invoices.reduce((sum, inv) => sum + (inv.paidAmount || 0), 0);
  const pending = invoices.reduce((sum, inv) => sum + (inv.totalAmount - inv.paidAmount), 0);

  const statusBreakdown = { Scheduled: 0, Completed: 0, Cancelled: 0 };
  statusAgg.forEach((s) => { statusBreakdown[s._id] = s.count; });

  // Revenue trend: last 6 months, grouped by paid invoices' createdAt
  const now = new Date();
  const months = [];
  for (let i = 5; i >= 0; i--) {
    const d = new Date(now.getFullYear(), now.getMonth() - i, 1);
    months.push({ year: d.getFullYear(), month: d.getMonth(), label: MONTHS[d.getMonth()] });
  }
  const totalsByKey = {};
  invoices.forEach((inv) => {
    const d = new Date(inv.createdAt);
    const key = `${d.getFullYear()}-${d.getMonth()}`;
    totalsByKey[key] = (totalsByKey[key] || 0) + (inv.paidAmount || 0);
  });
  const revenueTrend = months.map((m) => ({ label: m.label, total: totalsByKey[`${m.year}-${m.month}`] || 0 }));

  res.json({ patients, dentists, upcomingAppointments, revenue, pending, statusBreakdown, revenueTrend });
});

module.exports = router;
```

PART 2 — FRONTEND (React + Tailwind CSS)

The frontend lives in the **frontend/** folder and is built with React using Vite as the build tool. Every visible screen is a "page" component; small reusable pieces (buttons, modals, the sidebar) are "components"; and shared app-wide state (who's logged in, dark mode, toast popups) lives in "contexts". Styling is done entirely with Tailwind CSS utility classes directly in the JSX, rather than separate .css files.

frontend/src/main.jsx

WHAT THIS FILE DOES

The actual entry point of the React app — the very first file that runs in the browser. It finds the empty `<div id="root">` in `index.html` and tells React to render the whole `<App />` component tree inside it. `StrictMode` is a development-only wrapper that helps catch mistakes early.

THE CODE

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

frontend/src/App.jsx

WHAT THIS FILE DOES

Defines every URL/page in the app and wraps everything in the three shared context providers. This is the map of the whole website.

- `ThemeProvider`, `ToastProvider`, and `AuthProvider` wrap the entire app so dark mode, toast popups, and login state are available from *any* page without passing props down manually.
- `<Route path="/" element={<Landing />} />` etc. maps each URL to the component that should render there.
- Routes wrapped in `<PrivateRoute>` require any logged-in user; routes wrapped in `<AdminRoute>` (like `/staff`) additionally require the admin role. Landing, Login, and Register are left unwrapped since anyone should be able to see them.
- `<Route path="*" element={<NotFound />} />` is a catch-all: any URL that doesn't match anything above shows the 404 page.

THE CODE

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import { ThemeProvider } from "../context/ThemeContext";
import { ToastProvider } from "../context/ToastContext";
import { PatientAuthProvider } from "../context/PatientAuthContext";
import PrivateRoute from "../components/PrivateRoute";
import AdminRoute from "../components/AdminRoute";
import PatientPrivateRoute from "../components/PatientPrivateRoute";
import Landing from "../pages/Landing";
import Login from "../pages/Login";
import Register from "../pages/Register";
import Dashboard from "../pages/Dashboard";
import Patients from "../pages/Patients";
import PatientDetail from "../pages/PatientDetail";
import Dentists from "../pages/Dentists";
import Appointments from "../pages/Appointments";
import Billing from "../pages/Billing";
import Services from "../pages/Services";
import Staff from "../pages/Staff";
```

```

import Profile from "../pages/Profile";
import NotFound from "../pages/NotFound";
import PatientLogin from "../pages/patient/PatientLogin";
import PatientRegister from "../pages/patient/PatientRegister";
import PatientDashboard from "../pages/patient/PatientDashboard";
import PatientAppointments from "../pages/patient/PatientAppointments";
import PatientPayments from "../pages/patient/PatientPayments";
import PatientRecords from "../pages/patient/PatientRecords";

export default function App() {
  return (
    <ThemeProvider>
      <ToastProvider>
        <AuthProvider>
          <PatientAuthProvider>
            <BrowserRouter>
              <Routes>
                <Route path="/" element={<Landing />} />
                <Route path="/login" element={<Login />} />
                <Route path="/register" element={<Register />} />
                <Route path="/dashboard" element={<PrivateRoute><Dashboard /></PrivateRoute>} />
                <Route path="/patients" element={<PrivateRoute><Patients /></PrivateRoute>} />
                <Route path="/patients/:id" element={<PrivateRoute><PatientDetail /></PrivateRoute>} />
                <Route path="/dentists" element={<PrivateRoute><Dentists /></PrivateRoute>} />
                <Route path="/appointments" element={<PrivateRoute><Appointments /></PrivateRoute>} />
                <Route path="/billing" element={<PrivateRoute><Billing /></PrivateRoute>} />
                <Route path="/services" element={<PrivateRoute><Services /></PrivateRoute>} />
                <Route path="/staff" element={<AdminRoute><Staff /></AdminRoute>} />
                <Route path="/profile" element={<PrivateRoute><Profile /></PrivateRoute>} />

                {/* Patient self-service portal – separate login from staff */}
                <Route path="/patient-login" element={<PatientLogin />} />
                <Route path="/patient-register" element={<PatientRegister />} />
                <Route path="/patient/dashboard" element={<PatientPrivateRoute><PatientDashboard /></PatientPrivateRoute>} />
                <Route path="/patient/appointments" element={<PatientPrivateRoute><PatientAppointments /></PatientPrivateRoute>} />

              </Routes>
            </BrowserRouter>
          </PatientAuthProvider>
        </AuthProvider>
      </ToastProvider>
    </ThemeProvider>
  );
}

```

API Layer (frontend/src/api/)

frontend/src/api/axios.js

WHAT THIS FILE DOES

Creates one shared, pre-configured Axios instance that every page imports and reuses, instead of each page setting up its own HTTP client from scratch.

- `baseUrl` is read from an environment variable so the same code works whether the backend is running locally or deployed somewhere else.
- `api.interceptors.request.use(...)` is the key trick: it runs before *every* single request this instance makes, grabs the saved JWT token from `localStorage`, and automatically attaches it as an `Authorization: Bearer ...` header. This means no page ever has to remember to manually add the token — it's automatic.

THE CODE

```

import axios from "axios";

const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:5000/api";

const api = axios.create({ baseUrl: API_BASE });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
},

```

```

    return config;
  });

  export default api;

```

Shared State / Contexts (frontend/src/context/)

React Context is how data is shared across many components without passing it down through every layer manually ("prop drilling"). Each file below defines a Provider component (wrapped around the whole app in App.jsx) and a matching useXxx() hook that any page can call to read or update that shared state.

frontend/src/context/AuthContext.jsx

WHAT THIS FILE DOES

Manages who's logged in, everywhere in the app.

- On first load, it checks `localStorage` for a previously saved user so a page refresh doesn't log the person out.
- `login()` and `register()` call the backend, then save both the JWT token and the user object into `localStorage` (so the axios interceptor can find the token later) and update React state so the UI immediately reflects being logged in.
- `logout()` just clears both from storage and resets state to null; `PrivateRoute` then automatically redirects to `/login` because user becomes falsy.

THE CODE

```

import { createContext, useContext, useState } from "react";
import api from "../api/axios";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const stored = localStorage.getItem("user");
    return stored ? JSON.parse(stored) : null;
  });

  const login = async (email, password) => {
    const { data } = await api.post("/auth/login", { email, password });
    localStorage.setItem("token", data.token);
    localStorage.setItem("user", JSON.stringify(data.user));
    setUser(data.user);
  };

  const register = async (name, email, password, role) => {
    const { data } = await api.post("/auth/register", { name, email, password, role });
    localStorage.setItem("token", data.token);
    localStorage.setItem("user", JSON.stringify(data.user));
    setUser(data.user);
  };

  const logout = () => {
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, register, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);

```

frontend/src/context/ThemeContext.jsx

WHAT THIS FILE DOES

Manages dark mode. The dark boolean is initialised straight from `localStorage` so the theme survives a page refresh. Whenever it changes, a `useEffect` toggles the "dark" CSS class on the `<html>` element — Tailwind's `dark:` utility classes (like `dark:bg-slate-900`) only activate when that class is present.

THE CODE

```
import { createContext, useContext, useEffect, useState } from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [dark, setDark] = useState(() => localStorage.getItem("theme") === "dark");

  useEffect(() => {
    document.documentElement.classList.toggle("dark", dark);
    localStorage.setItem("theme", dark ? "dark" : "light");
  }, [dark]);

  return (
    <ThemeContext.Provider value={{ dark, toggle: () => setDark((d) => !d) }}>
      {children}
    </ThemeContext.Provider>
  );
}

export const useTheme = () => useContext(ThemeContext);
```

frontend/src/context/ToastContext.jsx

WHAT THIS FILE DOES

Manages the little pop-up messages ("Patient added", "Failed to load") that appear briefly in the bottom-right corner after most actions.

- `showToast(message, type)` adds a new toast object to an array with a unique id, then uses `setTimeout` to automatically remove that same toast 3 seconds later.
- The Provider renders the actual floating toast boxes itself (fixed-positioned, stacked with `space-y-2`), colored green/red/gray depending on the type, so any page can trigger a toast just by calling `showToast()` without rendering anything itself.

THE CODE

```
import { createContext, useContext, useState, useCallback } from "react";

const ToastContext = createContext(null);

export function ToastProvider({ children }) {
  const [toasts, setToasts] = useState([]);

  const showToast = useCallback((message, type = "success") => {
    const id = Date.now() + Math.random();
    setToasts((t) => [...t, { id, message, type }]);
    setTimeout(() => {
      setToasts((t) => t.filter((toast) => toast.id !== id));
    }, 3000);
  }, []);

  return (
    <ToastContext.Provider value={{ showToast }}>
      {children}
      <div className="fixed bottom-5 right-5 z-[100] space-y-2">
        {toasts.map((t) => (
          <div
            key={t.id}
            className={`px-4 py-3 rounded-lg shadow-lg text-sm text-white min-w-[220px] animate-[fadeIn_0.2s_ease-in] ${
              t.type === "error" ? "bg-red-500" : t.type === "info" ? "bg-slate-700" : "bg-green-600"
            }`}
          >
            {t.message}
          </div>
        ))}
      </div>
    </ToastContext.Provider>
  );
}
```

```
}  
  
export const useToast = () => useContext(ToastContext);
```

Reusable Components (frontend/src/components/)

frontend/src/components/PrivateRoute.jsx

WHAT THIS FILE DOES

A tiny wrapper component used in App.jsx around any page that should require login. It checks `useAuth()` for a user; if there isn't one, it renders `<Navigate to="/login" />` instead of the page, which redirects the browser. If there is a user, it just renders whatever page was passed in as children.

THE CODE

```
import { Navigate } from "react-router-dom";  
import { useAuth } from "../context/AuthContext";  
  
export default function PrivateRoute({ children }) {  
  const { user } = useAuth();  
  if (!user) return <Navigate to="/login" replace />;  
  return children;  
}
```

frontend/src/components/AdminRoute.jsx

WHAT THIS FILE DOES

The same idea as `PrivateRoute`, but stricter: it first checks the user is logged in at all (redirect to `/login` if not), then also checks `user.role === "admin"` (redirect to `/dashboard` if a regular staff member tries to visit an admin-only page like Staff Management).

THE CODE

```
import { Navigate } from "react-router-dom";  
import { useAuth } from "../context/AuthContext";  
  
export default function AdminRoute({ children }) {  
  const { user } = useAuth();  
  if (!user) return <Navigate to="/login" replace />;  
  if (user.role !== "admin") return <Navigate to="/dashboard" replace />;  
  return children;  
}
```

frontend/src/components/Layout.jsx

WHAT THIS FILE DOES

A very small structural component: it renders the `<Sidebar />` on the left and puts whatever page content is passed in as children on the right in a `<main>` area. Every logged-in page (Dashboard, Patients, etc.) wraps its content in `<Layout>...</Layout>` so the sidebar doesn't have to be rebuilt on every single page.

THE CODE

```
import Sidebar from "../Sidebar";  
  
export default function Layout({ children }) {  
  return (  
    <div className="flex bg-slate-50 dark:bg-slate-900 min-h-screen">  
      <Sidebar />  
      <main className="flex-1 p-8">{children}</main>  
    </div>  
  );  
}
```

frontend/src/components/Sidebar.jsx

WHAT THIS FILE DOES

The persistent left-hand navigation menu shown on every logged-in page.

- The links array drives the main nav items (Dashboard, Patients, Appointments, Dentists, Services, Billing) — NavLink automatically highlights whichever one matches the current URL.
- The "Staff" link only renders `{user?.role === "admin" && (...)}` — a conditional render so regular staff never even see a link to a page they're not allowed on.
- Also hosts the `<NotificationBell />` up top and `<ThemeToggle />` plus the Logout button at the bottom.

THE CODE

```
import { NavLink, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import ThemeToggle from "../ThemeToggle";
import NotificationBell from "../NotificationBell";

const links = [
  { to: "/dashboard", label: "Dashboard", icon: "■" },
  { to: "/patients", label: "Patients", icon: "■■■■■" },
  { to: "/appointments", label: "Appointments", icon: "■" },
  { to: "/dentists", label: "Dentists", icon: "■" },
  { to: "/services", label: "Services", icon: "■" },
  { to: "/billing", label: "Billing", icon: "■" },
];

export default function Sidebar() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <aside className="w-64 bg-slate-900 dark:bg-slate-950 text-white min-h-screen flex flex-col sticky top-0">
      <div className="p-5 border-b border-slate-700 flex items-start justify-between">
        <div>
          <h1 className="text-xl font-bold text-cyan-400">■ DentalCare</h1>
          <p className="text-xs text-slate-400 mt-1">Management System</p>
        </div>
        <NotificationBell />
      </div>
      <nav className="flex-1 p-3 space-y-1">
        {links.map((link) => (
          <NavLink
            key={link.to}
            to={link.to}
            className={({ isActive }) =>
              `flex items-center gap-3 px-4 py-2.5 rounded-lg text-sm transition ${
                isActive ? "bg-cyan-600 text-white" : "text-slate-300 hover:bg-slate-800"
              }`
          >
            <span>{link.icon}</span> {link.label}
          </NavLink>
        ))}
        {user?.role === "admin" && (
          <NavLink
            to="/staff"
            className={({ isActive }) =>
              `flex items-center gap-3 px-4 py-2.5 rounded-lg text-sm transition ${
                isActive ? "bg-cyan-600 text-white" : "text-slate-300 hover:bg-slate-800"
              }`
          >
            <span>■</span> Staff
          </NavLink>
        )}
        <NavLink
          to="/profile"
          className={({ isActive }) =>
            `flex items-center gap-3 px-4 py-2.5 rounded-lg text-sm transition ${
              isActive ? "bg-cyan-600 text-white" : "text-slate-300 hover:bg-slate-800"
            }`
        >

```



```

    >
    <span>■■■</span> Profile
  </NavLink>
</nav>
<div className="p-4 border-t border-slate-700">
  <ThemeToggle />
  <p className="text-sm text-slate-300">{user?.name}</p>
  <p className="text-xs text-slate-500 mb-3 capitalize">{user?.role}</p>
  <button
    onClick={handleLogout}
    className="w-full bg-slate-800 hover:bg-red-600 text-sm py-2 rounded-lg transition"
  >
    Logout
  </button>
</div>
</aside>
);
}

```

frontend/src/components/Modal.jsx

WHAT THIS FILE DOES

A generic reusable popup dialog. It's a semi-transparent full-screen overlay with a centered white (or dark, in dark mode) card. Every "Add Patient", "Add Dentist", "New Invoice" form in the app reuses this exact same component — they just pass in a different `title` and a different form as `children`, instead of each page building its own popup from scratch.

THE CODE

```

export default function Modal({ title, onClose, children }) {
  return (
    <div className="fixed inset-0 bg-black/40 flex items-center justify-center z-50 p-4">
      <div className="bg-white dark:bg-slate-800 rounded-xl shadow-xl w-full max-w-lg p-6 max-h-[90vh] overflow-y-auto">
        <div className="flex justify-between items-center mb-4">
          <h2 className="text-lg font-semibold text-slate-800 dark:text-white">{title}</h2>
          <button onClick={onClose} className="text-slate-400 hover:text-slate-700 dark:hover:text-slate-200 text-xl leading-none">
            &times;
          </button>
        </div>
        {children}
      </div>
    </div>
  );
}

```

frontend/src/components/StatCard.jsx

WHAT THIS FILE DOES

The small rounded box used on the Dashboard for things like "Total Patients: 42". Takes a label, a value, an emoji icon, and a background color as props — a classic example of a small, 'dumb'/presentational component that just displays whatever data it's given.

THE CODE

```

export default function StatCard({ label, value, icon, color }) {
  return (
    <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm p-5 flex items-center gap-4 border border-slate-100 dark:border-slate-700">
      <div className={`w-12 h-12 rounded-lg flex items-center justify-center text-2xl ${color} dark:bg-slate-700`}>
        {icon}
      </div>
      <div>
        <p className="text-sm text-slate-500 dark:text-slate-400">{label}</p>
        <p className="text-2xl font-bold text-slate-800 dark:text-white">{value}</p>
      </div>
    </div>
  );
}

```

frontend/src/components/ThemeToggle.jsx

WHAT THIS FILE DOES

The dark/light mode button shown in the sidebar. It reads `dark` and `toggle` from `useTheme()` and just calls `toggle()` on click, swapping the button's own label between "■ Dark Mode" and "■ Light Mode" depending on the current state.

THE CODE

```
import { useTheme } from "../context/ThemeContext";

export default function ThemeToggle() {
  const { dark, toggle } = useTheme();
  return (
    <button
      onClick={toggle}
      title="Toggle dark mode"
      className="w-full flex items-center justify-center gap-2 text-xs bg-slate-800 hover:bg-slate-700 text-slate-200 py-2 rounded-lg mb-2 transition"
    >
      {dark ? "■ Light Mode" : "■ Dark Mode"}
    </button>
  );
}
```

frontend/src/components/NotificationBell.jsx

WHAT THIS FILE DOES

The bell icon dropdown in the top-right of the sidebar.

- On mount, it fetches the unread count once and then re-fetches it every 15 seconds with `setInterval`, which is what makes new notifications appear without the person needing to refresh the page.
- A separate `useEffect` listens for clicks anywhere on the page (`document.addEventListener("mousedown", ...)`) and closes the dropdown if the click happened outside the bell — a common pattern for any dropdown menu.
- Clicking the bell for the first time also triggers `loadList()` to fetch the actual notification messages, not just the count, so it doesn't load data the user never opens.

THE CODE

```
import { useEffect, useRef, useState } from "react";
import api from "../api/axios";

export default function NotificationBell() {
  const [open, setOpen] = useState(false);
  const [notifications, setNotifications] = useState([]);
  const [count, setCount] = useState(0);
  const ref = useRef(null);

  const loadCount = () => api.get("/notifications/unread-count").then((res) => setCount(res.data.count)).catch(() => {});
  const loadList = () => api.get("/notifications").then((res) => setNotifications(res.data)).catch(() => {});

  useEffect(() => {
    loadCount();
    const interval = setInterval(loadCount, 15000);
    return () => clearInterval(interval);
  }, []);

  useEffect(() => {
    const onClickOutside = (e) => {
      if (ref.current && !ref.current.contains(e.target)) setOpen(false);
    };
    document.addEventListener("mousedown", onClickOutside);
    return () => document.removeEventListener("mousedown", onClickOutside);
  }, []);

  const toggleOpen = () => {
    if (!open) loadList();
    setOpen(o => !o);
  };

  const markAllRead = async () => {
```

```

    await api.put("/notifications/read-all");
    setCount(0);
    loadList();
  };

  const typeIcon = { appointment: "■", payment: "■", system: "■" };

  return (
    <div className="relative" ref={ref}>
      <button onClick={toggleOpen} className="relative w-9 h-9 flex items-center justify-center rounded-lg hover:bg-slate-800 text-slate-200">
        ■
        {count > 0 && (
          <span className="absolute -top-1 -right-1 bg-red-500 text-white text-[10px] font-bold w-4 h-4 rounded-full flex items-center justify-center">
            {count > 9 ? "9+" : count}
          </span>
        )}
      </button>

      {open && (
        <div className="absolute right-0 mt-2 w-80 bg-white dark:bg-slate-800 border border-slate-200 dark:border-slate-700 rounded-xl shadow-xl z-50 max-h-96 overflow-y-auto">
          <div className="flex justify-between items-center px-4 py-3 border-b border-slate-100 dark:border-slate-700">
            <span className="font-semibold text-sm text-slate-700 dark:text-white">Notifications</span>
            <button onClick={markAllRead} className="text-xs text-cyan-600 hover:underline">Mark all read</button>
          </div>
          {notifications.length === 0 && (
            <p className="text-sm text-slate-400 text-center py-6">No notifications yet</p>
          )}
          {notifications.map((n) => (
            <div key={n._id} className={`px-4 py-3 border-b border-slate-50 dark:border-slate-700 text-sm flex gap-2 ${!n.read ? "bg-cyan-50/50 dark:bg-cyan-950/30" : ""}`}>
              <span>{typeIcon[n.type]} || "■"</span>
              <div>
                <p className="text-slate-700 dark:text-slate-200">{n.message}</p>
                <p className="text-xs text-slate-400 mt-0.5">{new Date(n.createdAt).toLocaleString()}</p>
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  );
}

```

frontend/src/components/AppointmentCalendar.jsx

WHAT THIS FILE DOES

Renders the month-view calendar grid on the Appointments page (as an alternative to the plain list view).

- Uses the date-fns library to calculate the full grid of days to display (including the tail end of the previous month and start of the next, so every week row is full), recalculated only when the selected month changes thanks to useMemo.
- byDay groups the appointment list into a lookup object keyed by date string ("2026-08-30") so each calendar cell can instantly find its own appointments instead of looping through the whole list 42 times.
- Each day cell shows small colored dots (amber/green/red) for each appointment's status, and clicking a day with appointments opens a modal listing them via the onDayClick callback passed in by the Appointments page.

THE CODE

```

import { useMemo, useState } from "react";
import {
  startOfMonth, endOfMonth, startOfWeek, endOfWeek,
  eachDayOfInterval, format, isSameMonth, isSameDay, addMonths, subMonths,
} from "date-fns";

const statusDot = { Scheduled: "bg-amber-500", Completed: "bg-green-500", Cancelled: "bg-red-500" };

export default function AppointmentCalendar({ appointments, onDayClick }) {
  const [month, setMonth] = useState(new Date());

  const days = useMemo(() => {
    const start = startOfWeek(startOfMonth(month));
    const end = endOfWeek(endOfMonth(month));
    return eachDayOfInterval({ start, end });
  }, [month]);

```

```

const byDay = useMemo(() => {
  const map = {};
  appointments.forEach((a) => {
    const key = format(new Date(a.date), "yyyy-MM-dd");
    map[key] = map[key] || [];
    map[key].push(a);
  });
  return map;
}, [appointments]);

return (
  <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 p-4">
    <div className="flex justify-between items-center mb-4">
      <button onClick={() => setMonth((m) => subMonths(m, 1))} className="px-3 py-1 rounded-lg border dark:border-slate-600 text-sm text-slate-600 dark:text-slate-300 hover:bg-slate-50 dark:hover:bg-slate-700">< Prev</button>
      <h3 className="font-semibold text-slate-800 dark:text-white">{format(month, "MMMM yyyy")}</h3>
      <button onClick={() => setMonth((m) => addMonths(m, 1))} className="px-3 py-1 rounded-lg border dark:border-slate-600 text-sm text-slate-600 dark:text-slate-300 hover:bg-slate-50 dark:hover:bg-slate-700">Next ></button>
    </div>
    <div className="grid grid-cols-7 gap-1 text-center text-xs font-medium text-slate-400 mb-1">
      {["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"].map((d) => <div key={d}>{d}</div>)}
    </div>
    <div className="grid grid-cols-7 gap-1">
      {days.map((day) => {
        const key = format(day, "yyyy-MM-dd");
        const dayAppts = byDay[key] || [];
        const inMonth = isSameMonth(day, month);
        const today = isSameDay(day, new Date());
        return (
          <button
            key={key}
            onClick={() => dayAppts.length && onDayClick?.(dayAppts, day)}
            className={`min-h-[70px] p-1.5 rounded-lg border text-left align-top flex flex-col gap-0.5 transition ${
              inMonth ? "bg-white dark:bg-slate-800" : "bg-slate-50 dark:bg-slate-900 text-slate-300 dark:text-slate-600"
            } ${today ? "border-cyan-400" : "border-slate-100 dark:border-slate-700"} ${dayAppts.length ?
            "hover:border-cyan-400 cursor-pointer" : "cursor-default"}`}
            >
              <span className={`text-xs ${today ? "text-cyan-600 font-bold" : "text-slate-500 dark:text-slate-400"}`}>{format(day, "d")}</span>
              <div className="flex flex-wrap gap-0.5">
                {dayAppts.slice(0, 3).map((a) => (
                  <span key={a._id} className={`w-1.5 h-1.5 rounded-full ${statusDot[a.status]} || "bg-slate-400"}`
                    title={a.status} />
                ))}
                {dayAppts.length > 3 && <span className="text-[10px] text-slate-400">+{dayAppts.length - 3}</span>}
              </div>
            </button>
          );
        });
      }
    </div>
  </div>
);
}

```

Pages (frontend/src/pages/)

Each file here is one full screen of the app, matched to a URL in App.jsx. Most follow the same shape: load data from the API in a `useEffect`, keep it in `useState`, render it in a table or cards, and open a `<Modal>` with a form for adding/editing.

frontend/src/pages/Landing.jsx

WHAT THIS FILE DOES

The public marketing homepage shown at "/" to anyone, logged in or not. It's pure presentation — no API calls at all — built from three plain arrays (features, testimonials) mapped into cards, plus a hero section and a call-to-action footer, all linking to /login or /register.

THE CODE

```

import { Link } from "react-router-dom";

const features = [
  { icon: "■■■■■", title: "Patient Records", desc: "Store complete patient profiles, medical history and contact details in one place." },

```

```

{ icon: "■", title: "Smart Scheduling", desc: "Book, track and manage appointments across all your dentists with live status updates." },
{ icon: "■", title: "Dentist Directory", desc: "Manage specializations, experience and availability of every dentist on staff." },
{ icon: "■", title: "Treatment Catalog", desc: "Maintain a price list of services so billing is fast and consistent." },
{ icon: "■", title: "Billing & Invoices", desc: "Generate multi-item invoices, track payments and see outstanding balances instantly." },
{ icon: "■", title: "Analytics Dashboard", desc: "Visualize revenue trends and appointment activity with built-in charts." },
];

const testimonials = [
  { name: "Dr. Aisha Verma", role: "Clinic Owner", text: "Cut our front-desk paperwork in half. Everything from bookings to billing lives in one dashboard now." },
  { name: "Rohit Malhotra", role: "Clinic Manager", text: "The analytics view finally gives us a clear picture of monthly revenue and pending payments." },
];

export default function Landing() {
  return (
    <div className="bg-white dark:bg-slate-900 text-slate-800 dark:text-slate-100 min-h-screen">
      <nav className="flex items-center justify-between px-6 md:px-16 py-5 border-b border-slate-100 dark:border-slate-800">
        <span className="text-xl font-bold text-cyan-600 dark:text-cyan-400">■ DentalCare</span>
        <div className="space-x-3">
          <Link to="/patient-login" className="text-sm font-medium text-slate-600 dark:text-slate-300 hover:text-cyan-600">Patient Portal</Link>
          <Link to="/login" className="text-sm font-medium text-slate-600 dark:text-slate-300 hover:text-cyan-600">Staff Login</Link>
          <Link to="/register" className="bg-cyan-600 hover:bg-cyan-700 text-white text-sm font-medium px-4 py-2 rounded-lg">Get Started</Link>
        </div>
      </nav>

      <section className="text-center px-6 py-20 max-w-3xl mx-auto">
        <span className="inline-block bg-cyan-50 dark:bg-cyan-950 text-cyan-700 dark:text-cyan-300 text-xs font-medium px-3 py-1 rounded-full mb-5">Built for modern dental clinics</span>
        <h1 className="text-4xl md:text-5xl font-extrabold leading-tight mb-5">Run your dental clinic <span className="text-cyan-600 dark:text-cyan-400">without the paperwork</span></h1>
        <p className="text-slate-500 dark:text-slate-400 text-lg mb-8">One system for patients, appointments, dentists and billing – so your staff spend less time on admin and more time on patient care.</p>
        <div className="flex justify-center gap-3 flex-wrap">
          <Link to="/register" className="bg-cyan-600 hover:bg-cyan-700 text-white px-6 py-3 rounded-lg font-medium">Start Free</Link>
          <Link to="/login" className="border border-slate-200 dark:border-slate-700 px-6 py-3 rounded-lg font-medium hover:bg-slate-50 dark:hover:bg-slate-800">Staff Login</Link>
          <Link to="/patient-login" className="border border-cyan-200 dark:border-cyan-800 text-cyan-700 dark:text-cyan-400 px-6 py-3 rounded-lg font-medium hover:bg-cyan-50 dark:hover:bg-slate-800">Patient Portal</Link>
        </div>
      </section>

      <section className="px-6 md:px-16 py-16 bg-slate-50 dark:bg-slate-800/40">
        <h2 className="text-2xl font-bold text-center mb-10">Everything your front desk needs</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6 max-w-5xl mx-auto">
          {features.map((f) => (
            <div key={f.title} className="bg-white dark:bg-slate-800 p-6 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700">
              <div className="text-3xl mb-3">{f.icon}</div>
              <h3 className="font-semibold mb-1">{f.title}</h3>
              <p className="text-sm text-slate-500 dark:text-slate-400">{f.desc}</p>
            </div>
          ))}
        </div>
      </section>

      <section className="px-6 md:px-16 py-16 max-w-4xl mx-auto">
        <h2 className="text-2xl font-bold text-center mb-10">Trusted by clinic staff</h2>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
          {testimonials.map((t) => (
            <div key={t.name} className="bg-white dark:bg-slate-800 p-6 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700">
              <p className="text-slate-600 dark:text-slate-300 italic mb-4">{t.text}</p>
              <p className="font-semibold text-sm">{t.name}</p>
              <p className="text-xs text-slate-400">{t.role}</p>
            </div>
          ))}
        </div>
      </section>
    </div>
  );
}

```

```

    ))}
  </div>
</section>

<section className="text-center px-6 py-16 bg-cyan-600 text-white">
  <h2 className="text-2xl font-bold mb-3">Ready to modernize your clinic?</h2>
  <p className="mb-6 text-cyan-50">Create a free staff account and get started in minutes.</p>
  <Link to="/register" className="bg-white text-cyan-700 px-6 py-3 rounded-lg font-medium">
    Create Account
  </Link>
</section>

<footer className="text-center text-xs text-slate-400 py-6 border-t border-slate-100 dark:border-slate-800">
  © {new Date().getFullYear()} DentalCare Management System – Portfolio project.
</footer>
</div>
);
}

```

frontend/src/pages/Login.jsx

WHAT THIS FILE DOES

The staff login form. Keeps email/password in local state, and on submit calls `login()` from `AuthContext`. If that succeeds, `navigate("/dashboard")` takes the person straight to their dashboard. If it throws (wrong password, etc.), the error message from the backend's response is shown above the form.

THE CODE

```

import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    try {
      await login(email, password);
      navigate("/dashboard");
    } catch (err) {
      setError(err.response?.data?.message || "Login failed");
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-100 dark:bg-slate-900">
      <form onSubmit={handleSubmit} className="bg-white dark:bg-slate-800 p-8 rounded-xl shadow-md w-full max-w-sm">
        <h1 className="text-2xl font-bold text-center text-slate-800 dark:text-white mb-1">■ DentalCare</h1>
        <p className="text-center text-slate-500 text-sm mb-6">Staff Login</p>
        {error && <p className="bg-red-50 text-red-600 text-sm p-2 rounded mb-4">{error}</p>}
        <label className="text-sm text-slate-600">Email</label>
        <input
          type="email"
          required
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500"
        />
        <label className="text-sm text-slate-600">Password</label>
        <input
          type="password"
          required
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="w-full border rounded-lg p-2.5 mt-1 mb-6 outline-cyan-500"
        />
        <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium transition">
          Login
        </button>
        <p className="text-center text-sm text-slate-500 mt-4">
          No account? <Link to="/register" className="text-cyan-600 font-medium">Register</Link>
        </p>
      </form>
    </div>
  );
}

```

```

    </form>
  </div>
);
}

```

frontend/src/pages/Register.jsx

WHAT THIS FILE DOES

Almost identical to Login.jsx, but collects name, email, password, and a role dropdown (staff/admin) and calls `register()` instead of `login()`. In a real product the role dropdown wouldn't be exposed to the public — it's left here so you can easily create the first admin account to explore the admin-only features.

THE CODE

```

import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function Register() {
  const [form, setForm] = useState({ name: "", email: "", password: "", role: "staff" });
  const [error, setError] = useState("");
  const { register } = useAuth();
  const navigate = useNavigate();

  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    try {
      await register(form.name, form.email, form.password, form.role);
      navigate("/dashboard");
    } catch (err) {
      setError(err.response?.data?.message || "Registration failed");
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-100 dark:bg-slate-900">
      <form onSubmit={handleSubmit} className="bg-white dark:bg-slate-800 p-8 rounded-xl shadow-md w-full max-w-sm">
        <h1 className="text-2xl font-bold text-center text-slate-800 dark:text-white mb-1">■ DentalCare</h1>
        <p className="text-center text-slate-500 text-sm mb-6">Create Staff Account</p>
        {error && <p className="bg-red-50 text-red-600 text-sm p-2 rounded mb-4">{error}</p>}
        <label className="text-sm text-slate-600">Full Name</label>
        <input name="name" required value={form.name} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500" />
        <label className="text-sm text-slate-600">Email</label>
        <input type="email" name="email" required value={form.email} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500" />
        <label className="text-sm text-slate-600">Password</label>
        <input type="password" name="password" required value={form.password} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500" />
        <label className="text-sm text-slate-600">Role</label>
        <select name="role" value={form.role} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-6 outline-cyan-500">
          <option value="staff">Staff</option>
          <option value="admin">Admin</option>
        </select>
        <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium transition">
          Register
        </button>
        <p className="text-center text-sm text-slate-500 mt-4">
          Already have an account? <Link to="/login" className="text-cyan-600 font-medium">Login</Link>
        </p>
      </form>
    </div>
  );
}

```

frontend/src/pages/Dashboard.jsx

WHAT THIS FILE DOES

The home screen after logging in. Fetches two things on load: overall /dashboard/stats and today's appointment list, then renders four stat cards, a revenue line chart, an appointment-status pie chart, a pending-payments card, and a live "today's appointments" list.

- Both charts use the Recharts library: LineChart for the 6-month revenue trend and PieChart for the Scheduled/Completed/Cancelled breakdown, each wrapped in a ResponsiveContainer so they resize to fit their card.
- pieData is derived from stats.statusBreakdown (an object like {Scheduled: 5, Completed: 2, Cancelled: 1}) by converting it into the array-of-objects shape Recharts expects with Object.entries().

THE CODE

```
import { useEffect, useState } from "react";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer, PieChart, Pie, Cell, CartesianGrid } from "recharts";
import Layout from "../components/Layout";
import StatCard from "../components/StatCard";
import api from "../api/axios";

const COLORS = { Scheduled: "#f59e0b", Completed: "#22c55e", Cancelled: "#ef4444" };

export default function Dashboard() {
  const [stats, setStats] = useState(null);
  const [today, setToday] = useState([]);
  const [error, setError] = useState("");

  useEffect(() => {
    api.get("/dashboard/stats").then((res) => setStats(res.data)).catch(() => setError("Could not load stats. Make sure backend & MongoDB are running.));
    api.get("/appointments/today/list").then((res) => setToday(res.data)).catch(() => {});
  }, []);

  const pieData = stats
    ? Object.entries(stats.statusBreakdown).map(([name, value]) => ({ name, value }))
    : [];

  return (
    <Layout>
      <h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">Dashboard</h1>
      <p className="text-slate-500 dark:text-slate-400 mb-6">Overview of your dental clinic</p>

      {error && <p className="bg-yellow-50 text-yellow-700 p-3 rounded-lg mb-6 text-sm">{error}</p>}

      <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-6">
        <StatCard label="Total Patients" value={stats?.patients ?? "-"} icon="■■■■" color="bg-cyan-50" />
        <StatCard label="Total Dentists" value={stats?.dentists ?? "-"} icon="■" color="bg-purple-50" />
        <StatCard label="Upcoming Appointments" value={stats?.upcomingAppointments ?? "-"} icon="■" color="bg-amber-50" />
        <StatCard label="Revenue Collected" value={stats ? `■$${stats.revenue}` : "-"} icon="■" color="bg-green-50" />
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-4 mb-6">
        <div className="lg:col-span-2 bg-white dark:bg-slate-800 rounded-xl shadow-sm p-6 border border-slate-100 dark:border-slate-700">
          <h2 className="font-semibold text-slate-800 dark:text-white mb-4">Revenue Trend (last 6 months)</h2>
          <ResponsiveContainer width="100%" height={220}>
            <LineChart data={stats?.revenueTrend || []}>
              <CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0" />
              <XAxis dataKey="label" tick={{ fontSize: 12 }} />
              <YAxis tick={{ fontSize: 12 }} />
              <Tooltip />
              <Line type="monotone" dataKey="total" stroke="#0891b2" strokeWidth={2} dot={{ r: 3 }} />
            </LineChart>
          </ResponsiveContainer>
        </div>

        <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm p-6 border border-slate-100 dark:border-slate-700">
          <h2 className="font-semibold text-slate-800 dark:text-white mb-4">Appointments by Status</h2>
          <ResponsiveContainer width="100%" height={220}>
            <PieChart>
              <Pie data={pieData} dataKey="value" nameKey="name" innerRadius={45} outerRadius={75}>
                {pieData.map((entry) => (
                  <Cell key={entry.name} fill={COLORS[entry.name] || "#94a3b8"} />
                ))}
              </Pie>
              <Tooltip />
            </PieChart>
          </ResponsiveContainer>
          <div className="flex justify-center gap-4 mt-2 text-xs">
            {Object.entries(COLORS).map(([name, color]) => (
              <span key={name} className="flex items-center gap-1 text-slate-500 dark:text-slate-400">
                <span className="w-2 h-2 rounded-full inline-block" style={{ background: color }} /> {name}
              </span>
            ))}
          </div>
        </div>
      </div>
    </Layout>
  );
}
```



```

    ))}
  </div>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
  <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm p-6 border border-slate-100 dark:border-slate-700">
    <h2 className="font-semibold text-slate-800 dark:text-white mb-2">Pending Payments</h2>
    <p className="text-3xl font-bold text-red-500">{stats ? `■${stats.pending}` : "-"}</p>
    <p className="text-sm text-slate-500 dark:text-slate-400 mt-1">Total amount yet to be collected from patients</p>
  </div>

  <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm p-6 border border-slate-100 dark:border-slate-700">
    <h2 className="font-semibold text-slate-800 dark:text-white mb-3">Today's Appointments</h2>
    {today.length === 0 && <p className="text-sm text-slate-400">No appointments scheduled for today.</p>}
    <ul className="space-y-2">
      {today.map((a) => (
        <li key={a._id} className="flex justify-between text-sm border-b border-slate-100 dark:border-slate-700 pb-2">
          <span className="text-slate-700 dark:text-slate-200">{a.patient?.name} — {a.dentist?.name}</span>
          <span className="text-slate-400">{a.time}</span>
        </li>
      ))}
    </ul>
  </div>
</div>
</Layout>
);
}

```

frontend/src/pages/Patients.jsx

WHAT THIS FILE DOES

The main patient list page, and the most feature-complete list view in the app.

- Search, gender filter, and pagination are all controlled inputs whose state changes trigger the `useEffect` to reload from the API with new query parameters — the actual filtering/pagination logic lives on the backend, this page just sends the right parameters.
- Changing the search box or gender filter also resets page back to 1, so you don't end up stuck on "page 4" of a search that only has 1 page of results.
- "Export CSV" requests the file as a `blob`, wraps it in a temporary object URL, and simulates a click on an invisible `<a download>` link — the standard browser trick for triggering a file download from JavaScript.
- Each patient's name is a `<Link to={`/patients/${p._id}`}>`, which is how you get to the detailed `PatientDetail` page.
- The Delete button only renders `{isAdmin && (...)}` — regular staff can add/edit patients but not delete them.

THE CODE

```

import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import Layout from "../components/Layout";
import Modal from "../components/Modal";
import api from "../api/axios";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";

const empty = { name: "", age: "", gender: "Male", phone: "", email: "", address: "", medicalHistory: "", bloodGroup: "" };

export default function Patients() {
  const [patients, setPatients] = useState([]);
  const [total, setTotal] = useState(0);
  const [pages, setPages] = useState(1);
  const [page, setPage] = useState(1);
  const [search, setSearch] = useState("");
  const [gender, setGender] = useState("");
  const [showModal, setShowModal] = useState(false);
  const [form, setForm] = useState(empty);
  const [editId, setEditId] = useState(null);
  const { user } = useAuth();
  const { showToast } = useToast();
  const isAdmin = user?.role === "admin";

  const load = () => {
    api

```

```

    .get("/patients", { params: { search, gender, page, limit: 8 } })
    .then((res) => {
      setPatients(res.data.patients);
      setTotal(res.data.total);
      setPages(res.data.pages);
    })
    .catch(() => showToast("Failed to load patients", "error"));
  });

useEffect(load, [page, search, gender]);

const openAdd = () => { setForm(empty); setEditId(null); setShowModal(true); };
const openEdit = (p) => { setForm(p); setEditId(p._id); setShowModal(true); };
const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    if (editId) await api.put(`/patients/${editId}`, form);
    else await api.post("/patients", form);
    setShowModal(false);
    load();
    showToast(editId ? "Patient updated" : "Patient added");
  } catch (err) {
    showToast(err.response?.data?.message || "Save failed", "error");
  }
};

const handleDelete = async (id) => {
  if (!confirm("Delete this patient?")) return;
  await api.delete(`/patients/${id}`);
  load();
  showToast("Patient deleted", "info");
};

const exportCsv = async () => {
  try {
    const res = await api.get("/patients/export/csv", { responseType: "blob" });
    const url = window.URL.createObjectURL(new Blob([res.data]));
    const link = document.createElement("a");
    link.href = url;
    link.setAttribute("download", "patients.csv");
    document.body.appendChild(link);
    link.click();
    link.remove();
    showToast("CSV exported");
  } catch {
    showToast("Export failed", "error");
  }
};

return (
  <Layout>
    <div className="flex flex-wrap justify-between items-center gap-3 mb-6">
      <div>
        <h1 className="text-2xl font-bold text-slate-800 dark:text-white">Patients</h1>
        <p className="text-slate-500 dark:text-slate-400">{total} total patient{total !== 1 && "s"}</p>
      </div>
      <div className="flex gap-2">
        <button onClick={exportCsv} className="border border-slate-200 dark:border-slate-600 text-slate-600 dark:text-slate-300 px-4 py-2 rounded-lg text-sm font-medium">
          ■ Export CSV
        </button>
        <button onClick={openAdd} className="bg-cyan-600 hover:bg-cyan-700 text-white px-4 py-2 rounded-lg text-sm font-medium">
          + Add Patient
        </button>
      </div>
    </div>

    <div className="flex flex-wrap gap-3 mb-4">
      <input
        placeholder="Search by name, phone or email..."
        value={search}
        onChange={(e) => { setPage(1); setSearch(e.target.value); }}
        className="flex-1 min-w-[220px] border dark:border-slate-600 dark:bg-slate-800 dark:text-white rounded-lg p-2.5 text-sm"
      />
      <select value={gender} onChange={(e) => { setPage(1); setGender(e.target.value); }} className="border dark:border-slate-600 dark:bg-slate-800 dark:text-white rounded-lg p-2.5 text-sm">
        <option value="">All Genders</option>
        <option>Male</option>
        <option>Female</option>
        <option>Other</option>
      </select>
    </div>
  </Layout>
);

```

```

    </select>
  </div>

  <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700
  overflow-x-auto">
    <table className="w-full text-sm">
      <thead className="bg-slate-50 dark:bg-slate-700 text-slate-500 dark:text-slate-300 text-left">
        <tr>
          <th className="p-3">Name</th>
          <th className="p-3">Age</th>
          <th className="p-3">Gender</th>
          <th className="p-3">Phone</th>
          <th className="p-3">Blood Group</th>
          <th className="p-3">Actions</th>
        </tr>
      </thead>
      <tbody>
        {patients.map((p) => (
          <tr key={p._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
            <td className="p-3 font-medium">
              <Link to={`/patients/${p._id}`} className="text-cyan-700 dark:text-cyan-400 hover:underline">{p.name}</Link>
            </td>
            <td className="p-3">{p.age}</td>
            <td className="p-3">{p.gender}</td>
            <td className="p-3">{p.phone}</td>
            <td className="p-3">{p.bloodGroup}</td>
            <td className="p-3 space-x-2">
              <button onClick={() => openEdit(p)} className="text-cyan-600 hover:underline">Edit</button>
              {isAdmin && <button onClick={() => handleDelete(p._id)} className="text-red-500
              hover:underline">Delete</button>}
            </td>
          </tr>
        ))}
        {patients.length === 0 && (
          <tr><td colspan="6" className="p-6 text-center text-slate-400">No patients found</td></tr>
        )}
      </tbody>
    </table>
  </div>

  {pages > 1 && (
    <div className="flex justify-center items-center gap-2 mt-4">
      <button disabled={page <= 1} onClick={() => setPage((p) => p - 1)} className="px-3 py-1.5 border rounded-lg text-sm
      disabled:opacity-40 dark:border-slate-600 dark:text-slate-300">Prev</button>
      <span className="text-sm text-slate-500 dark:text-slate-400">Page {page} of {pages}</span>
      <button disabled={page >= pages} onClick={() => setPage((p) => p + 1)} className="px-3 py-1.5 border rounded-lg
      text-sm disabled:opacity-40 dark:border-slate-600 dark:text-slate-300">Next</button>
    </div>
  )}

  {showModal && (
    <Modal title={editId ? "Edit Patient" : "Add Patient"} onClose={() => setShowModal(false)}>
      <form onSubmit={handleSubmit} className="space-y-3">
        <input name="name" placeholder="Full name" required value={form.name} onChange={handleChange} className="w-full
        border rounded-lg p-2.5" />
        <div className="grid grid-cols-2 gap-3">
          <input name="age" type="number" placeholder="Age" value={form.age} onChange={handleChange} className="border
          rounded-lg p-2.5" />
          <select name="gender" value={form.gender} onChange={handleChange} className="border rounded-lg p-2.5">
            <option>Male</option>
            <option>Female</option>
            <option>Other</option>
          </select>
        </div>
        <input name="phone" placeholder="Phone number" required value={form.phone} onChange={handleChange} className="w-full
        border rounded-lg p-2.5" />
        <input name="email" placeholder="Email" value={form.email} onChange={handleChange} className="w-full border
        rounded-lg p-2.5" />
        <input name="address" placeholder="Address" value={form.address} onChange={handleChange} className="w-full border
        rounded-lg p-2.5" />
        <input name="bloodGroup" placeholder="Blood group" value={form.bloodGroup} onChange={handleChange} className="w-full
        border rounded-lg p-2.5" />
        <textarea name="medicalHistory" placeholder="Medical history / allergies" value={form.medicalHistory}
        onChange={handleChange} className="w-full border rounded-lg p-2.5" rows="3" />
        <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium">
          {editId ? "Update Patient" : "Add Patient"}
        </button>
      </form>
    </Modal>
  )}
</Layout>
);
}

```

frontend/src/pages/PatientDetail.jsx

WHAT THIS FILE DOES

The full profile page for one patient, opened by clicking their name in the Patients list. Uses a tabbed layout (Info / Appointments / Invoices / Medical Records) all on one URL, switching what's shown with a `tab` state variable instead of separate routes.

- On load it calls three endpoints in parallel: `/patients/:id/full` (patient + appointments + invoices together), `/records/patient/:id` (medical records), and `/dentists` (to populate the dentist dropdown in the "add record" form).
- The Medical Records tab's "Add Record" form optionally uploads a file first (POST `/upload` with `FormData`), gets back a URL, and only then creates the medical record itself with that URL attached — two separate API calls chained together in `submitRecord()`.
- Attachment links are built by stripping the trailing `/api` off the axios base URL and appending the file's relative `/uploads/filename.png` path, since uploaded files are served as static files directly by Express, not through the `/api` prefix.

THE CODE

```
import { useEffect, useState } from "react";
import { useParams, Link } from "react-router-dom";
import Layout from "../components/Layout";
import Modal from "../components/Modal";
import api from "../api/axios";
import { useToast } from "../context/ToastContext";

const emptyRecord = { diagnosis: "", treatmentDone: "", prescription: "", notes: "", dentist: "" };

export default function PatientDetail() {
  const { id } = useParams();
  const [data, setData] = useState(null);
  const [tab, setTab] = useState("info");
  const [records, setRecords] = useState([]);
  const [dentists, setDentists] = useState([]);
  const [showRecordModal, setShowRecordModal] = useState(false);
  const [recordForm, setRecordForm] = useState(emptyRecord);
  const [file, setFile] = useState(null);
  const [uploading, setUploading] = useState(false);
  const { showToast } = useToast();

  const loadRecords = () => api.get(`/records/patient/${id}`).then((res) => setRecords(res.data));

  useEffect(() => {
    api.get(`/patients/${id}/full`).then((res) => setData(res.data));
    loadRecords();
    api.get("/dentists").then((res) => setDentists(res.data));
  }, [id]);

  if (!data) return <Layout><p className="text-slate-400">Loading...</p></Layout>;

  const { patient, appointments, invoices } = data;

  const tabs = [
    { key: "info", label: "Info" },
    { key: "appointments", label: `Appointments (${appointments.length})` },
    { key: "invoices", label: `Invoices (${invoices.length})` },
    { key: "records", label: `Medical Records (${records.length})` },
  ];

  const handleRecordChange = (e) => setRecordForm({ ...recordForm, [e.target.name]: e.target.value });

  const submitRecord = async (e) => {
    e.preventDefault();
    try {
      let attachments = [];
      if (file) {
        setUploading(true);
        const fd = new FormData();
        fd.append("file", file);
        const res = await api.post("/upload", fd, { headers: { "Content-Type": "multipart/form-data" } });
        attachments = [{ filename: res.data.filename, url: res.data.url }];
      }
      await api.post("/records", { ...recordForm, patient: id, attachments });
      setShowRecordModal(false);
      setRecordForm(emptyRecord);
    }
  };
}
```

```

    setFile(null);
    loadRecords();
    showToast("Medical record added");
  } catch (err) {
    showToast(err.response?.data?.message || "Failed to add record", "error");
  } finally {
    setUploading(false);
  }
};

const deleteRecord = async (recordId) => {
  if (!confirm("Delete this medical record?")) return;
  await api.delete(`/records/${recordId}`);
  loadRecords();
  showToast("Record deleted", "info");
};

return (
  <Layout>
    <Link to="/patients" className="text-sm text-cyan-600 hover:underline">&larr; Back to Patients</Link>
    <div className="flex items-center gap-4 mt-3 mb-6">
      <div className="w-14 h-14 rounded-full bg-cyan-100 text-cyan-700 flex items-center justify-center text-xl font-bold">
        {patient.name?.[0]?.toUpperCase()}
      </div>
      <div>
        <h1 className="text-2xl font-bold text-slate-800 dark:text-white">{patient.name}</h1>
        <p className="text-slate-500 dark:text-slate-400">{patient.phone} • {patient.gender}, {patient.age} yrs</p>
      </div>
    </div>

    <div className="flex justify-between items-center mb-6 border-b border-slate-200 dark:border-slate-700">
      <div className="flex gap-2">
        {tabs.map((t) => (
          <button
            key={t.key}
            onClick={() => setTab(t.key)}
            className={`px-4 py-2 text-sm font-medium border-b-2 -mb-px ${
              tab === t.key ? "border-cyan-600 text-cyan-600" : "border-transparent text-slate-500 dark:text-slate-400"
            }`}
          >
            {t.label}
          </button>
        ))}
      </div>
      {tab === "records" && (
        <button onClick={() => setShowRecordModal(true)} className="mb-2 bg-cyan-600 hover:bg-cyan-700 text-white px-3 py-1.5 rounded-lg text-sm font-medium">
          + Add Record
        </button>
      )}
    </div>

    {tab === "info" && (
      <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 p-6 grid grid-cols-2 gap-4 text-sm max-w-2xl">
        <div><span className="text-slate-400">Email</span><p className="text-slate-700 dark:text-slate-200">{patient.email || "-"}</p></div>
        <div><span className="text-slate-400">Blood Group</span><p className="text-slate-700 dark:text-slate-200">{patient.bloodGroup || "-"}</p></div>
        <div className="col-span-2"><span className="text-slate-400">Address</span><p className="text-slate-700 dark:text-slate-200">{patient.address || "-"}</p></div>
        <div className="col-span-2"><span className="text-slate-400">Medical History / Allergies</span><p className="text-slate-700 dark:text-slate-200">{patient.medicalHistory || "-"}</p></div>
      </div>
    )}

    {tab === "appointments" && (
      <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 overflow-x-auto">
        <table className="w-full text-sm">
          <thead className="bg-slate-50 dark:bg-slate-700 text-slate-500 dark:text-slate-300 text-left">
            <tr><th className="p-3">Date</th><th className="p-3">Time</th><th className="p-3">Dentist</th><th className="p-3">Reason</th><th className="p-3">Status</th></tr>
          </thead>
          <tbody>
            {appointments.map((a) => (
              <tr key={a._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
                <td className="p-3">{new Date(a.date).toLocaleDateString()}</td>
                <td className="p-3">{a.time}</td>
                <td className="p-3">{a.dentist?.name}</td>
                <td className="p-3">{a.reason}</td>
                <td className="p-3">{a.status}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
  </Layout>
);

```

```

        {appointments.length === 0 && <tr><td colspan="5" className="p-6 text-center text-slate-400">No appointments
yet</td></tr>}
      </tbody>
    </table>
  </div>
)}

{tab === "invoices" && (
  <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700
overflow-x-auto">
    <table className="w-full text-sm">
      <thead className="bg-slate-50 dark:bg-slate-700 text-slate-500 dark:text-slate-300 text-left">
        <tr><th className="p-3">Date</th><th className="p-3">Total</th><th className="p-3">Paid</th><th
className="p-3">Status</th></tr>
      </thead>
      <tbody>
        {invoices.map((inv) => (
          <tr key={inv._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
            <td className="p-3">{new Date(inv.createdAt).toLocaleDateString()}</td>
            <td className="p-3">■{inv.totalAmount}</td>
            <td className="p-3">■{inv.paidAmount}</td>
            <td className="p-3">■{inv.status}</td>
          </tr>
        ))}
        {invoices.length === 0 && <tr><td colspan="4" className="p-6 text-center text-slate-400">No invoices
yet</td></tr>}
      </tbody>
    </table>
  </div>
)}

{tab === "records" && (
  <div className="space-y-3">
    {records.length === 0 && (
      <p className="text-slate-400 text-center py-6 bg-white dark:bg-slate-800 rounded-xl border border-slate-100
dark:border-slate-700 p-5">
        No medical records yet.
      </p>
    )}
    {records.map((r) => (
      <div key={r._id} className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100
dark:border-slate-700 p-5">
        <div className="flex justify-between items-start mb-2">
          <div>
            <p className="font-semibold text-slate-800 dark:text-white">{new Date(r.visitDate).toLocaleDateString()}</p>
            {r.dentist?.name && <p className="text-xs text-cyan-600">Dr. {r.dentist.name} •
{r.dentist.specialization}</p>}
          </div>
          <button onClick={() => deleteRecord(r._id)} className="text-red-500 text-sm hover:underline">Delete</button>
        </div>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-3 text-sm">
          {r.diagnosis && <div><span className="text-slate-400">Diagnosis</span><p className="text-slate-700
dark:text-slate-200">{r.diagnosis}</p></div>}
          {r.treatmentDone && <div><span className="text-slate-400">Treatment Done</span><p className="text-slate-700
dark:text-slate-200">{r.treatmentDone}</p></div>}
          {r.prescription && <div className="md:col-span-2"><span className="text-slate-400">Prescription</span><p
className="text-slate-700 dark:text-slate-200">{r.prescription}</p></div>}
          {r.notes && <div className="md:col-span-2"><span className="text-slate-400">Notes</span><p
className="text-slate-700 dark:text-slate-200">{r.notes}</p></div>}
        </div>
        {r.attachments?.length > 0 && (
          <div className="mt-3 flex flex-wrap gap-2">
            {r.attachments.map((a, i) => (
              <a
                key={i}
                href={`\${api.defaults.baseURL.replace(/\//api$/, "")}\${a.url}`}
                target="_blank"
                rel="noreferrer"
                className="text-xs bg-slate-100 dark:bg-slate-700 text-cyan-700 dark:text-cyan-400 px-2 py-1 rounded-md
hover:underline"
                >
                  ■ {a.filename || "Attachment"}
                </a>
            ))}
          </div>
        )}
      </div>
    )}
  </div>
)}

{showRecordModal && (
  <Modal title="Add Medical Record" onClose={() => setShowRecordModal(false)}>
    <form onSubmit={submitRecord} className="space-y-3">
      <select name="dentist" value={recordForm.dentist} onChange={handleRecordChange} className="w-full border rounded-lg

```

```

p-2.5">
    <option value="">Select dentist (optional)</option>
    {dentists.map((d) => <option key={d._id} value={d._id}>{d.name}</option>)}
  </select>
  <input name="diagnosis" placeholder="Diagnosis" value={recordForm.diagnosis} onChange={handleRecordChange}
className="w-full border rounded-lg p-2.5" />
  <input name="treatmentDone" placeholder="Treatment done" value={recordForm.treatmentDone}
onChange={handleRecordChange} className="w-full border rounded-lg p-2.5" />
  <textarea name="prescription" placeholder="Prescription" value={recordForm.prescription}
onChange={handleRecordChange} className="w-full border rounded-lg p-2.5" rows="2" />
  <textarea name="notes" placeholder="Additional notes" value={recordForm.notes} onChange={handleRecordChange}
className="w-full border rounded-lg p-2.5" rows="2" />
  <div>
    <label className="text-sm text-slate-600 dark:text-slate-300">Attach X-ray / document (image or PDF,
optional)</label>
    <input type="file" accept="image/*,.pdf" onChange={(e) => setFile(e.target.files[0])} className="w-full border
rounded-lg p-2 mt-1 text-sm" />
  </div>
  <button disabled={uploading} className="w-full bg-cyan-600 hover:bg-cyan-700 disabled:opacity-60 text-white py-2.5
rounded-lg font-medium">
    {uploading ? "Uploading..." : "Save Record"}
  </button>
</form>
</Modal>
  )}
</Layout>
);
}

```

frontend/src/pages/Dentists.jsx

WHAT THIS FILE DOES

A simple grid of dentist cards with Add/Edit/Delete, following the exact same load/openAdd/openEdit/handleSubmit/handleDelete pattern used throughout the app. Good file to compare side-by-side with backend/routes/dentists.js to see the full request/response loop for a simple CRUD feature.

THE CODE

```

import { useEffect, useState } from "react";
import Layout from "../components/Layout";
import Modal from "../components/Modal";
import api from "../api/axios";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";

const empty = { name: "", specialization: "", phone: "", email: "", experienceYears: "" };

export default function Dentists() {
  const [dentists, setDentists] = useState([]);
  const [showModal, setShowModal] = useState(false);
  const [form, setForm] = useState(empty);
  const [editId, setEditId] = useState(null);
  const { user } = useAuth();
  const { showToast } = useToast();
  const isAdmin = user?.role === "admin";

  const load = () => api.get("/dentists").then((res) => setDentists(res.data));
  useEffect(load, []);

  const openAdd = () => { setForm(empty); setEditId(null); setShowModal(true); };
  const openEdit = (d) => { setForm(d); setEditId(d._id); setShowModal(true); };
  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      if (editId) await api.put(`/dentists/${editId}`, form);
      else await api.post("/dentists", form);
      setShowModal(false);
      load();
      showToast(editId ? "Dentist updated" : "Dentist added");
    } catch (err) {
      showToast(err.response?.data?.message || "Save failed", "error");
    }
  };

  const handleDelete = async (id) => {
    if (!confirm("Delete this dentist?")) return;
  };
}

```

```

    await api.delete(`/dentists/${id}`);
    load();
    showToast("Dentist deleted", "info");
  };

  return (
    <Layout>
      <div className="flex justify-between items-center mb-6">
        <div>
          <h1 className="text-2xl font-bold text-slate-800 dark:text-white">Dentists</h1>
          <p className="text-slate-500 dark:text-slate-400">Manage clinic dentists / staff</p>
        </div>
        <button onClick={openAdd} className="bg-cyan-600 hover:bg-cyan-700 text-white px-4 py-2 rounded-lg text-sm font-medium">
          + Add Dentist
        </button>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        {dentists.map((d) => (
          <div key={d._id} className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 p-5">
            <h3 className="font-semibold text-slate-800 dark:text-white">{d.name}</h3>
            <p className="text-sm text-cyan-600 dark:text-cyan-400">{d.specialization}</p>
            <p className="text-sm text-slate-500 dark:text-slate-400 mt-2">■ {d.phone}</p>
            <p className="text-sm text-slate-500 dark:text-slate-400">■ {d.email}</p>
            <p className="text-sm text-slate-500 dark:text-slate-400">■ {d.experienceYears} yrs experience</p>
            <div className="mt-3 space-x-3">
              <button onClick={() => openEdit(d)} className="text-cyan-600 text-sm hover:underline">Edit</button>
              {isAdmin && <button onClick={() => handleDelete(d._id)} className="text-red-500 text-sm hover:underline">Delete</button>}
            </div>
          </div>
        ))}
        {dentists.length === 0 && <p className="text-slate-400">No dentists added yet</p>}
      </div>

      {showModal && (
        <Modal title={editId ? "Edit Dentist" : "Add Dentist"} onClose={() => setShowModal(false)}>
          <form onSubmit={handleSubmit} className="space-y-3">
            <input name="name" placeholder="Full name" required value={form.name} onChange={handleChange} className="w-full border rounded-lg p-2.5" />
            <input name="specialization" placeholder="Specialization (e.g. Orthodontist)" value={form.specialization} onChange={handleChange} className="w-full border rounded-lg p-2.5" />
            <input name="phone" placeholder="Phone number" value={form.phone} onChange={handleChange} className="w-full border rounded-lg p-2.5" />
            <input name="email" placeholder="Email" value={form.email} onChange={handleChange} className="w-full border rounded-lg p-2.5" />
            <input name="experienceYears" type="number" placeholder="Years of experience" value={form.experienceYears} onChange={handleChange} className="w-full border rounded-lg p-2.5" />
            <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium">
              {editId ? "Update Dentist" : "Add Dentist"}
            </button>
          </form>
        </Modal>
      )}
    </Layout>
  );
}

```

frontend/src/pages/Appointments.jsx

WHAT THIS FILE DOES

Manages booking and viewing appointments, with a toggle between a plain list view and the visual month calendar view.

- view state ('list' or 'calendar') decides which component renders below the toggle buttons — both views share the exact same appointments data loaded once at the top.
- Clicking a day on the calendar sets `dayModal`, which opens a `<Modal>` listing every appointment booked that day.
- "Complete" / "Cancel" buttons only show up while an appointment's status is still "Scheduled", and call `updateStatus()` which sends a small PUT request with just the new status.

THE CODE

```

import { useEffect, useState } from "react";
import Layout from "../components/Layout";

```



```

import Modal from "../components/Modal";
import AppointmentCalendar from "../components/AppointmentCalendar";
import api from "../api/axios";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";

const empty = { patient: "", dentist: "", date: "", time: "", reason: "", status: "Scheduled" };

export default function Appointments() {
  const [appointments, setAppointments] = useState([]);
  const [patients, setPatients] = useState([]);
  const [dentists, setDentists] = useState([]);
  const [showModal, setShowModal] = useState(false);
  const [form, setForm] = useState(empty);
  const [view, setView] = useState("list");
  const [dayModal, setDayModal] = useState(null);
  const { user } = useAuth();
  const { showToast } = useToast();
  const isAdmin = user?.role === "admin";

  const load = () => api.get("/appointments").then((res) => setAppointments(res.data));

  useEffect(() => {
    load();
    api.get("/patients", { params: { limit: 1000 } }).then((res) => setPatients(res.data.patients || res.data));
    api.get("/dentists").then((res) => setDentists(res.data));
  }, []);

  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await api.post("/appointments", form);
      setShowModal(false);
      setForm(empty);
      load();
      showToast("Appointment booked");
    } catch (err) {
      showToast(err.response?.data?.message || "Booking failed", "error");
    }
  };

  const updateStatus = async (id, status) => {
    await api.put(`/appointments/${id}`, { status });
    load();
    showToast(`Marked as ${status}`);
  };

  const handleDelete = async (id) => {
    if (!confirm("Delete this appointment?")) return;
    await api.delete(`/appointments/${id}`);
    load();
    showToast("Appointment deleted", "info");
  };

  const statusColor = { Scheduled: "bg-amber-100 text-amber-700", Completed: "bg-green-100 text-green-700", Cancelled: "bg-red-100 text-red-700" };

  return (
    <Layout>
      <div className="flex justify-between items-center mb-6">
        <div>
          <h1 className="text-2xl font-bold text-slate-800 dark:text-white">Appointments</h1>
          <p className="text-slate-500 dark:text-slate-400">Schedule and track patient visits</p>
        </div>
        <div className="flex items-center gap-3">
          <div className="flex bg-slate-100 dark:bg-slate-800 rounded-lg p-1">
            <button onClick={() => setView("list")} className={`px-3 py-1.5 rounded-md text-sm font-medium ${view === "list" ? "bg-white dark:bg-slate-700 shadow-sm text-slate-800 dark:text-white" : "text-slate-500 dark:text-slate-400"}`}>List</button>
            <button onClick={() => setView("calendar")} className={`px-3 py-1.5 rounded-md text-sm font-medium ${view === "calendar" ? "bg-white dark:bg-slate-700 shadow-sm text-slate-800 dark:text-white" : "text-slate-500 dark:text-slate-400"}`}>Calendar</button>
          </div>
          <button onClick={() => setShowModal(true)} className="bg-cyan-600 hover:bg-cyan-700 text-white px-4 py-2 rounded-lg text-sm font-medium">
            + New Appointment
          </button>
        </div>
      </div>

      {view === "calendar" && (
        <AppointmentCalendar appointments={appointments} onDayClick={({ list, day }) => setDayModal({ list, day })} />
      )}
    </Layout>
  );
}

```

```

    })

    {view === "list" && (
    <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700
overflow-x-auto">
      <table className="w-full text-sm">
        <thead className="bg-slate-50 dark:bg-slate-700 text-slate-500 dark:text-slate-300 text-left">
          <tr>
            <th className="p-3">Patient</th>
            <th className="p-3">Dentist</th>
            <th className="p-3">Date</th>
            <th className="p-3">Time</th>
            <th className="p-3">Reason</th>
            <th className="p-3">Status</th>
            <th className="p-3">Actions</th>
          </tr>
        </thead>
        <tbody>
          {appointments.map((a) => (
            <tr key={a._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
              <td className="p-3">{a.patient?.name}</td>
              <td className="p-3">{a.dentist?.name}</td>
              <td className="p-3">{new Date(a.date).toLocaleDateString()}</td>
              <td className="p-3">{a.time}</td>
              <td className="p-3">{a.reason}</td>
              <td className="p-3">
                <span className={`px-2 py-1 rounded-full text-xs font-medium ${statusColor[a.status]}`}>{a.status}</span>
              </td>
              <td className="p-3 space-x-2">
                {a.status === "Scheduled" && (
                  <>
                    <button onClick={() => updateStatus(a._id, "Completed")} className="text-green-600
hover:underline">Complete</button>
                    <button onClick={() => updateStatus(a._id, "Cancelled")} className="text-amber-600
hover:underline">Cancel</button>
                  </>
                )}
                {isAdmin && <button onClick={() => handleDelete(a._id)} className="text-red-500
hover:underline">Delete</button>}
              </td>
            </tr>
          ))}
          {appointments.length === 0 && (
            <tr><td colspan="7" className="p-6 text-center text-slate-400">No appointments yet</td></tr>
          )}
        </tbody>
      </table>
    </div>
  )}

  {dayModal && (
    <Modal title={dayModal.day.toString()} onClose={() => setDayModal(null)}>
      <div className="space-y-3">
        {dayModal.list.map((a) => (
          <div key={a._id} className="flex justify-between items-center border border-slate-100 dark:border-slate-700
rounded-lg p-3">
            <div>
              <p className="font-medium text-slate-800 dark:text-white">{a.patient?.name} – {a.dentist?.name}</p>
              <p className="text-xs text-slate-500 dark:text-slate-400">{a.time} • {a.reason || "No reason given"}</p>
            </div>
            <span className={`px-2 py-1 rounded-full text-xs font-medium ${statusColor[a.status]}`}>{a.status}</span>
          </div>
        ))}
      </div>
    </Modal>
  )}

  {showModal && (
    <Modal title="New Appointment" onClose={() => setShowModal(false)}>
      <form onSubmit={handleSubmit} className="space-y-3">
        <select name="patient" required value={form.patient} onChange={handleChange} className="w-full border rounded-lg
p-2.5">
          <option value="">Select patient</option>
          {patients.map((p) => <option key={p._id} value={p._id}>{p.name}</option>)}
        </select>
        <select name="dentist" required value={form.dentist} onChange={handleChange} className="w-full border rounded-lg
p-2.5">
          <option value="">Select dentist</option>
          {dentists.map((d) => <option key={d._id} value={d._id}>{d.name}</option>)}
        </select>
        <div className="grid grid-cols-2 gap-3">
          <input name="date" type="date" required value={form.date} onChange={handleChange} className="border rounded-lg
p-2.5" />
          <input name="time" type="time" required value={form.time} onChange={handleChange} className="border rounded-lg

```

```

    p-2.5" />
    </div>
    <input name="reason" placeholder="Reason for visit" value={form.reason} onChange={handleChange} className="w-full
border rounded-lg p-2.5" />
    <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium">
      Book Appointment
    </button>
  </form>
</Modal>
)}
</Layout>
);
}

```

frontend/src/pages/Billing.jsx

WHAT THIS FILE DOES

Handles creating invoices and downloading them as PDF files.

- The invoice form supports multiple treatment line items; typing a treatment name that matches an existing entry in the Services price list (via the `<data list>` autocomplete) automatically fills in that service's price with `updateItem()`.
- `downloadInvoice()` uses the jsPDF library entirely in the browser — no server involved — to draw a clinic header, invoice details, and a table of line items (via the `jspdf-autotable` plugin), then triggers `doc.save(...)` which downloads the generated PDF straight to the user's computer.
- The invoice's status (Paid / Partially Paid / Unpaid) is calculated on the frontend right before submitting, by comparing the amount paid now against the total.

THE CODE

```

import { useEffect, useState } from "react";
import Layout from "../components/Layout";
import Modal from "../components/Modal";
import api from "../api/axios";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";
import jsPDF from "jspdf";
import autoTable from "jspdf-autotable";

export default function Billing() {
  const [invoices, setInvoices] = useState([]);
  const [patients, setPatients] = useState([]);
  const [services, setServices] = useState([]);
  const [showModal, setShowModal] = useState(false);
  const [patient, setPatient] = useState("");
  const [items, setItems] = useState([{ treatment: "", cost: "" }]);
  const [paidAmount, setPaidAmount] = useState(0);
  const { user } = useAuth();
  const { showToast } = useToast();
  const isAdmin = user?.role === "admin";

  const load = () => api.get("/invoices").then((res) => setInvoices(res.data));

  useEffect(() => {
    load();
    api.get("/patients", { params: { limit: 1000 } }).then((res) => setPatients(res.data.patients || res.data));
    api.get("/services").then((res) => setServices(res.data));
  }, []);

  const addItem = () => setItems([...items, { treatment: "", cost: "" }]);
  const updateItem = (i, field, value) => {
    const copy = [...items];
    copy[i][field] = value;
    if (field === "treatment") {
      const match = services.find((s) => s.name === value);
      if (match) copy[i].cost = match.price;
    }
    setItems(copy);
  };
  const removeItem = (i) => setItems(items.filter((_, idx) => idx !== i));

  const total = items.reduce((sum, it) => sum + (Number(it.cost) || 0), 0);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const status = paidAmount >= total ? "Paid" : paidAmount > 0 ? "Partially Paid" : "Unpaid";

```

```

try {
  await api.post("/invoices", { patient, items, totalAmount: total, paidAmount, status });
  setShowModal(false);
  setPatient("");
  setItems([{ treatment: "", cost: "" }]);
  setPaidAmount(0);
  load();
  showToast("Invoice created");
} catch (err) {
  showToast(err.response?.data?.message || "Failed to create invoice", "error");
}
};

const handleDelete = async (id) => {
  if (!confirm("Delete this invoice?")) return;
  await api.delete(`/invoices/${id}`);
  load();
  showToast("Invoice deleted", "info");
};

const downloadInvoice = (inv) => {
  const doc = new jsPDF();
  doc.setFontSize(18);
  doc.setTextColor(8, 145, 178);
  doc.text("DentalCare Management System", 14, 18);
  doc.setFontSize(11);
  doc.setTextColor(60, 60, 60);
  doc.text("Invoice", 14, 26);

  doc.setFontSize(10);
  doc.text(`Invoice ID: ${inv._id}`, 14, 36);
  doc.text(`Date: ${new Date(inv.createdAt).toLocaleDateString()}`, 14, 42);
  doc.text(`Patient: ${inv.patient?.name || "-"}`, 14, 48);
  doc.text(`Status: ${inv.status}`, 14, 54);

  autoTable(doc, {
    startY: 62,
    head: [["Treatment", "Cost (■)"]],
    body: inv.items.map((it) => [it.treatment, it.cost.toString()]),
    theme: "striped",
    headStyles: { fillColor: [8, 145, 178] },
  });

  const finalY = doc.lastAutoTable.finalY + 10;
  doc.setFontSize(11);
  doc.text(`Total: ■${inv.totalAmount}`, 14, finalY);
  doc.text(`Paid: ■${inv.paidAmount}`, 14, finalY + 6);
  doc.text(`Balance: ■${inv.totalAmount - inv.paidAmount}`, 14, finalY + 12);

  doc.save(`invoice-${inv.patient?.name || "patient"}-${inv._id.slice(-6)}.pdf`);
};

const statusColor = { Paid: "bg-green-100 text-green-700", "Partially Paid": "bg-amber-100 text-amber-700", Unpaid:
  "bg-red-100 text-red-700" };

return (
  <Layout>
    <div className="flex justify-between items-center mb-6">
      <div>
        <h1 className="text-2xl font-bold text-slate-800 dark:text-white">Billing & Invoices</h1>
        <p className="text-slate-500 dark:text-slate-400">Track treatments and payments</p>
      </div>
      <button onClick={() => setShowModal(true)} className="bg-cyan-600 hover:bg-cyan-700 text-white px-4 py-2 rounded-lg text-sm font-medium">
        + New Invoice
      </button>
    </div>

    <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 overflow-x-auto">
      <table className="w-full text-sm">
        <thead className="bg-slate-50 dark:bg-slate-700 text-slate-500 dark:text-slate-300 text-left">
          <tr>
            <th className="p-3">Patient</th>
            <th className="p-3">Total</th>
            <th className="p-3">Paid</th>
            <th className="p-3">Balance</th>
            <th className="p-3">Status</th>
            <th className="p-3">Actions</th>
          </tr>
        </thead>
        <tbody>
          {invoices.map((inv) => (
            <tr key={inv._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">

```

```
 {inv.patient?.name}</td>  ■{inv.totalAmount}</td>  ■{inv.paidAmount}</td>  ■{inv.totalAmount - inv.paidAmount}</td>  <span className={`px-2 py-1 rounded-full text-xs font-medium ${statusColor[inv.status]}`}>{inv.status}</span></td>   | | | | | |
```

frontend/src/pages/Services.jsx

WHAT THIS FILE DOES

Manages the treatment price catalog. Services are grouped by category using `reduce()` to build an object like `{Preventive: [...], Surgery: [...]}`, then rendered as one card grid per category with a heading. Add/Edit/Delete buttons are hidden entirely for non-admins (`isAdmin && (...)`), matching the admin-only restriction enforced on the backend for the same actions.

THE CODE

```

import { useEffect, useState } from "react";
import Layout from "../../components/Layout";
import Modal from "../../components/Modal";

```

```

import api from "../api/axios";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";

const empty = { name: "", category: "General", price: "", durationMinutes: 30, description: "" };

export default function Services() {
  const [services, setServices] = useState([]);
  const [showModal, setShowModal] = useState(false);
  const [form, setForm] = useState(empty);
  const [editId, setEditId] = useState(null);
  const { user } = useAuth();
  const { showToast } = useToast();
  const isAdmin = user?.role === "admin";

  const load = () => api.get("/services").then((res) => setServices(res.data));
  useEffect(load, []);

  const openAdd = () => { setForm(empty); setEditId(null); setShowModal(true); };
  const openEdit = (s) => { setForm(s); setEditId(s._id); setShowModal(true); };
  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      if (editId) await api.put(`/services/${editId}`, form);
      else await api.post("/services", form);
      setShowModal(false);
      load();
      showToast(editId ? "Service updated" : "Service added");
    } catch (err) {
      showToast(err.response?.data?.message || "Save failed", "error");
    }
  };

  const handleDelete = async (id) => {
    if (!confirm("Delete this service?")) return;
    await api.delete(`/services/${id}`);
    load();
    showToast("Service deleted", "info");
  };

  const grouped = services.reduce((acc, s) => {
    acc[s.category] = acc[s.category] || [];
    acc[s.category].push(s);
    return acc;
  }, {});

  return (
    <Layout>
      <div className="flex justify-between items-center mb-6">
        <div>
          <h1 className="text-2xl font-bold text-slate-800 dark:text-white">Treatments & Services</h1>
          <p className="text-slate-500 dark:text-slate-400">Price list used when creating invoices</p>
        </div>
        {isAdmin && (
          <button onClick={openAdd} className="bg-cyan-600 hover:bg-cyan-700 text-white px-4 py-2 rounded-lg text-sm font-medium">
            + Add Service
          </button>
        )}
      </div>

      {Object.keys(grouped).length === 0 && <p className="text-slate-400">No services added yet.</p>}

      {Object.entries(grouped).map(([category, list]) => (
        <div key={category} className="mb-6">
          <h2 className="text-sm font-semibold text-slate-500 dark:text-slate-400 uppercase tracking-wide mb-2">{category}</h2>
          <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
            {list.map((s) => (
              <div key={s._id} className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 p-5">
                <div className="flex justify-between items-start">
                  <h3 className="font-semibold text-slate-800 dark:text-white">{s.name}</h3>
                  <span className="text-cyan-600 dark:text-cyan-400 font-bold">■{s.price}</span>
                </div>
                <p className="text-xs text-slate-400 mt-1">{s.durationMinutes} min</p>
                {s.description && <p className="text-sm text-slate-500 dark:text-slate-400 mt-2">{s.description}</p>}
                {isAdmin && (
                  <div className="mt-3 space-x-3">
                    <button onClick={() => openEdit(s)} className="text-cyan-600 text-sm hover:underline">Edit</button>
                    <button onClick={() => handleDelete(s._id)} className="text-red-500 text-sm hover:underline">Delete</button>
                  </div>
                )}
              </div>
            ))}
          </div>
        </div>
      ))}
    </Layout>
  );
}

```

```

    </div>
  )}
</div>
</div>
)}

{showModal && (
  <Modal title={editId ? "Edit Service" : "Add Service"} onClose={() => setShowModal(false)}>
    <form onSubmit={handleSubmit} className="space-y-3">
      <input name="name" placeholder="Service name" required value={form.name} onChange={handleChange} className="w-full border rounded-lg p-2.5" />
      <input name="category" placeholder="Category (e.g. Preventive, Surgery)" value={form.category}
onChange={handleChange} className="w-full border rounded-lg p-2.5" />
      <div className="grid grid-cols-2 gap-3">
        <input name="price" type="number" placeholder="Price (■)" required value={form.price} onChange={handleChange}
className="border rounded-lg p-2.5" />
        <input name="durationMinutes" type="number" placeholder="Duration (min)" value={form.durationMinutes}
onChange={handleChange} className="border rounded-lg p-2.5" />
      </div>
      <textarea name="description" placeholder="Description" value={form.description} onChange={handleChange}
className="w-full border rounded-lg p-2.5" rows="2" />
      <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium">
        {editId ? "Update Service" : "Add Service"}
      </button>
    </form>
  </Modal>
)}
</Layout>
);
}

```

frontend/src/pages/Staff.jsx

WHAT THIS FILE DOES

The admin-only staff management table (only reachable through the AdminRoute-protected /staff URL).

- `toggleRole()` and `toggleActive()` both call the same generic PUT `/users/:id` backend route, just with different fields flipped — `role` between admin/staff, or `isActive` between true/false.
- The delete handler checks `u._id === currentUser.id` on the frontend too (as a friendlier version of the same safety check that exists on the backend) so an admin gets an immediate, clear toast instead of a generic server error if they try to remove themselves.

THE CODE

```

import { useEffect, useState } from "react";
import Layout from "../components/Layout";
import api from "../api/axios";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";

export default function Staff() {
  const [users, setUsers] = useState([]);
  const { user: currentUser } = useAuth();
  const { showToast } = useToast();

  const load = () => api.get("/users").then((res) => setUsers(res.data));
  useEffect(load, []);

  const toggleRole = async (u) => {
    const role = u.role === "admin" ? "staff" : "admin";
    await api.put(`/users/${u._id}`, { role, isActive: u.isActive });
    load();
    showToast(`${u.name} is now ${role}`);
  };

  const toggleActive = async (u) => {
    await api.put(`/users/${u._id}`, { role: u.role, isActive: !u.isActive });
    load();
    showToast(u.isActive ? `${u.name} deactivated` : `${u.name} activated`);
  };

  const handleDelete = async (u) => {
    if (u._id === currentUser.id) return showToast("You can't delete your own account", "error");
    if (!confirm(`Remove ${u.name}?`)) return;
    await api.delete(`/users/${u._id}`);
    load();
  };

```

```

    showToast("Staff member removed", "info");
  };

  return (
    <Layout>
      <h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">Staff Management</h1>
      <p className="text-slate-500 dark:text-slate-400 mb-6">Admin-only: manage staff accounts and roles</p>

      <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700
overflow-x-auto">
        <table className="w-full text-sm">
          <thead className="bg-slate-50 dark:bg-slate-700 text-slate-500 dark:text-slate-300 text-left">
            <tr>
              <th className="p-3">Name</th>
              <th className="p-3">Email</th>
              <th className="p-3">Role</th>
              <th className="p-3">Status</th>
              <th className="p-3">Actions</th>
            </tr>
          </thead>
          <tbody>
            {users.map((u) => (
              <tr key={u._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
                <td className="p-3 font-medium">{u.name} {u._id === currentUser.id && <span className="text-xs
text-cyan-500">(you)</span>}</td>
                <td className="p-3">{u.email}</td>
                <td className="p-3 capitalize">{u.role}</td>
                <td className="p-3">
                  <span className={`px-2 py-1 rounded-full text-xs font-medium ${u.isActive ? "bg-green-100 text-green-700" :
"bg-red-100 text-red-700"}`>
                    {u.isActive ? "Active" : "Inactive"}
                  </span>
                </td>
                <td className="p-3 space-x-2">
                  <button onClick={() => toggleRole(u)} className="text-cyan-600 hover:underline">Make {u.role === "admin" ?
"Staff" : "Admin"}</button>
                  <button onClick={() => toggleActive(u)} className="text-amber-600 hover:underline">{u.isActive ? "Deactivate"
: "Activate"}</button>
                  <button onClick={() => handleDelete(u)} className="text-red-500 hover:underline">Remove</button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </Layout>
  );
}

```

frontend/src/pages/Profile.jsx

WHAT THIS FILE DOES

A personal settings page showing the logged-in user's name/role and some notification preference checkboxes. This page is currently front-end only for the preferences (they show a toast on save but aren't sent anywhere) — a good example of a page that's intentionally simple and left as a place to wire up the already-built `PUT /users/me` and `PUT /users/me/password` backend endpoints if you want to extend it further.

THE CODE

```

import { useState } from "react";
import Layout from "../components/Layout";
import { useAuth } from "../context/AuthContext";
import { useToast } from "../context/ToastContext";

export default function Profile() {
  const { user } = useAuth();
  const { showToast } = useToast();
  const [notifEmail, setNotifEmail] = useState(true);
  const [notifSms, setNotifSms] = useState(false);

  const handleSave = (e) => {
    e.preventDefault();
    showToast("Preferences saved");
  };

  return (
    <Layout>

```



```

<h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">Profile & Settings</h1>
<p className="text-slate-500 dark:text-slate-400 mb-6">Your account information</p>

<div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 p-6
max-w-lg">
  <div className="flex items-center gap-4 mb-6">
    <div className="w-16 h-16 rounded-full bg-cyan-100 text-cyan-700 flex items-center justify-center text-2xl font-bold">
      {user?.name?.[0]?.toUpperCase()}
    </div>
    <div>
      <p className="font-semibold text-slate-800 dark:text-white">{user?.name}</p>
      <p className="text-sm text-slate-500 dark:text-slate-400 capitalize">{user?.role}</p>
    </div>
  </div>

  <form onSubmit={handleSave} className="space-y-4">
    <div>
      <label className="text-sm text-slate-600 dark:text-slate-300">Full name</label>
      <input defaultValue={user?.name} className="w-full border dark:border-slate-600 dark:bg-slate-700 dark:text-white
rounded-lg p-2.5 mt-1" />
    </div>
    <div className="flex items-center justify-between">
      <span className="text-sm text-slate-600 dark:text-slate-300">Email notifications for appointments</span>
      <input type="checkbox" checked={notifEmail} onChange={() => setNotifEmail(!notifEmail)} className="w-4 h-4" />
    </div>
    <div className="flex items-center justify-between">
      <span className="text-sm text-slate-600 dark:text-slate-300">SMS reminders</span>
      <input type="checkbox" checked={notifSms} onChange={() => setNotifSms(!notifSms)} className="w-4 h-4" />
    </div>
    <button className="w-full bg-cyan-600 hover:bg-cyan-700 text-white py-2.5 rounded-lg font-medium">
      Save Changes
    </button>
  </form>
</div>
</Layout>
);
}

```

frontend/src/pages/NotFound.jsx

WHAT THIS FILE DOES

The catch-all 404 page shown for any URL that doesn't match a real route (see the `path="*" route in App.jsx`). Simple, friendly messaging with a button back to the homepage — no API calls, no state.

THE CODE

```

import { Link } from "react-router-dom";

export default function NotFound() {
  return (
    <div className="min-h-screen flex flex-col items-center justify-center bg-slate-50 dark:bg-slate-900 text-center px-4">
      <p className="text-6xl mb-4">■</p>
      <h1 className="text-3xl font-bold text-slate-800 dark:text-white mb-2">404 – Page not found</h1>
      <p className="text-slate-500 dark:text-slate-400 mb-6">The page you're looking for doesn't exist.</p>
      <Link to="/" className="bg-cyan-600 hover:bg-cyan-700 text-white px-5 py-2.5 rounded-lg font-medium">
        Go Home
      </Link>
    </div>
  );
}

```

PART 3 — THE PATIENT PORTAL (self-service add-on)

On top of the staff-facing app, there's a second, completely separate login area where patients can register themselves, view their own appointment/payment/medical history, download PDFs, and receive automatic emails. It shares the same database and the same Express server as the staff app, but keeps its own login session so a patient's browser tab and a staff member's browser tab never get mixed up.

Backend additions

backend/models/Patient.js

WHAT THIS FILE DOES

Updated for the portal: the Patient model now also stores a hashed password (only set once a patient registers for portal access), a hasPortalAccess flag, and emailVerified. It uses the exact same bcrypt hashing pattern as the staff User model.

- `select: false` on the password field means normal queries like `Patient.find()` never accidentally return the password hash — the login route has to explicitly ask for it with `.select("+password")`.

THE CODE

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const patientSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    age: Number,
    gender: { type: String, enum: ["Male", "Female", "Other"] },
    phone: { type: String, required: true },
    email: { type: String, unique: true, sparse: true }, // sparse: many old records may have no email
    address: String,
    medicalHistory: String,
    bloodGroup: String,
    photo: String,

    // --- Patient portal login (added for self-service portal) ---
    password: { type: String, select: false }, // only set once the patient registers for portal access
    hasPortalAccess: { type: Boolean, default: false },
    emailVerified: { type: Boolean, default: false },
  },
  { timestamps: true }
);

patientSchema.methods.comparePassword = function (candidate) {
  return bcrypt.compare(candidate, this.password);
};

patientSchema.pre("save", async function (next) {
  if (!this.isModified("password") || !this.password) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

module.exports = mongoose.model("Patient", patientSchema);
```

backend/models/Appointment.js

WHAT THIS FILE DOES

Gained one new field: `reminderSent` (boolean, default false). This is how the reminder-email cron job avoids emailing the same patient twice about the same appointment — once a reminder goes out, this flips to true and that appointment is skipped on the next run.

THE CODE

```
const mongoose = require("mongoose");
```

```
const appointmentSchema = new mongoose.Schema(
  {
    patient: { type: mongoose.Schema.Types.ObjectId, ref: "Patient", required: true },
    dentist: { type: mongoose.Schema.Types.ObjectId, ref: "Dentist", required: true },
    date: { type: Date, required: true },
    time: { type: String, required: true },
    reason: String,
    status: {
      type: String,
      enum: ["Scheduled", "Completed", "Cancelled"],
      default: "Scheduled",
    },
    notes: String,
    reminderSent: { type: Boolean, default: false },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Appointment", appointmentSchema);
```

backend/middleware/patientOnly.js

WHAT THIS FILE DOES

The patient-portal equivalent of `adminOnly`. It checks `req.user.role === "patient"` and rejects the request with 403 otherwise. Every route in `portal.js` uses this, which guarantees a staff member's login token can never be used to browse a patient's private data, and a patient's token can never touch staff-only routes.

THE CODE

```
// Guards the patient-portal routes so a staff/admin JWT can't be used to view
// patient-portal data, and vice versa — the two account types are kept separate.
module.exports = function patientOnly(req, res, next) {
  if (!req.user || req.user.role !== "patient") {
    return res.status(403).json({ message: "This endpoint is for patient portal accounts only" });
  }
  next();
};
```

backend/config/mailer.js

WHAT THIS FILE DOES

A small wrapper around the Nodemailer library that every email-sending feature calls into.

- `getTransporter()` reads SMTP connection details from environment variables (`host/port/user/pass`). If they're not set, it returns null instead of crashing — the app is fully usable with email turned off.
- `sendMail()` is written to **never throw**: if sending fails (or isn't configured), it logs a warning and resolves to `false`. This is deliberate — booking an appointment or creating an invoice should always succeed even if the confirmation email happens to fail.

THE CODE

```
const nodemailer = require("nodemailer");

// The transporter reads its connection details from environment variables so
// real credentials are never hardcoded. See backend/.env.example for the
// variables you need to set (SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, FROM_EMAIL).
let transporter = null;

function getTransporter() {
  if (transporter) return transporter;
  if (!process.env.SMTP_HOST || !process.env.SMTP_USER) {
    return null; // no email configured yet — features that use it will just skip sending
  }
  transporter = nodemailer.createTransport({
    host: process.env.SMTP_HOST,
    port: Number(process.env.SMTP_PORT) || 587,
    secure: Number(process.env.SMTP_PORT) === 465,
    auth: {
      user: process.env.SMTP_USER,
      pass: process.env.SMTP_PASS,
    },
  });
}
```

```

    },
  });
  return transporter;
}

// Sends an email. Never throws – logs a warning and resolves false instead,
// so a failed/misconfigured email never breaks the feature that triggered it
// (e.g. booking an appointment should succeed even if the confirmation email fails).
async function sendMail({ to, subject, html, attachments }) {
  const t = getTransporter();
  if (!t) {
    console.warn(`[mailer] SMTP not configured – skipped email "${subject}" to ${to}`);
    return false;
  }
  try {
    await t.sendMail({
      from: process.env.FROM_EMAIL || '"DentalCare Clinic" <no-reply@dentalcare.example>',
      to,
      subject,
      html,
      attachments,
    });
    return true;
  } catch (err) {
    console.error(`[mailer] Failed to send "${subject}" to ${to}:`, err.message);
    return false;
  }
}

module.exports = { sendMail };

```

backend/utils/emailTemplates.js

WHAT THIS FILE DOES

Plain functions that return an HTML string for each type of email (welcome, appointment confirmed, appointment reminder, payment receipt, medical report). A shared wrapper () helper wraps every template in the same clinic-branded header/footer so all emails look consistent, without needing a full templating engine for a project this size.

THE CODE

```

// Simple inline-HTML email templates. Kept as small functions returning a
// string so they're easy to preview/edit without a templating engine.

const wrapper = (title, bodyHtml) => `
<div style="font-family: Arial, sans-serif; max-width: 520px; margin: 0 auto; padding: 24px; color:#1e293b;">
  <div style="background:#0e7490; color:#fff; padding:18px 24px; border-radius:10px 10px 0 0;">
    <h2 style="margin:0; font-size:20px;">■ DentalCare Clinic</h2>
  </div>
  <div style="border:1px solid #e2e8f0; border-top:none; padding:24px; border-radius:0 0 10px 10px;">
    <h3 style="color:#0e7490; margin-top:0;">${title}</h3>
    ${bodyHtml}
    <p style="color:#94a3b8; font-size:12px; margin-top:28px;">This is an automated message from DentalCare Clinic. Please do
    not reply directly to this email.</p>
  </div>
</div>`;

function registrationEmail(name) {
  return wrapper("Welcome to DentalCare!", `
    <p>Hi ${name}</p>
    <p>Your patient portal account has been created successfully. You can now log in anytime to view your
    appointment history, payment history, and medical reports.</p>`);
}

function appointmentConfirmationEmail({ patientName, dentistName, date, time, reason }) {
  return wrapper("Appointment Confirmed ■", `
    <p>Hi ${patientName}</p>
    <p>Your appointment has been booked:</p>
    <table style="width:100%; border-collapse:collapse; margin:12px 0;">
      <tr><td style="padding:6px 0; color:#64748b;">Dentist</td><td style="padding:6px 0;"><b>Dr. ${dentistName}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Date</td><td style="padding:6px 0;"><b>${date}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Time</td><td style="padding:6px 0;"><b>${time}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Reason</td><td style="padding:6px 0;"><b>${reason || "-"}</td></tr>
    </table>
    <p>We look forward to seeing you!</p>`);
}

```

```

function appointmentReminderEmail({ patientName, dentistName, date, time }) {
  return wrapper("Appointment Reminder ■", `
    <p>Hi ${patientName},</p>
    <p>This is a friendly reminder about your upcoming appointment tomorrow:</p>
    <table style="width:100%; border-collapse:collapse; margin:12px 0;">
      <tr><td style="padding:6px 0; color:#64748b;">Dentist</td><td style="padding:6px 0;"><b>Dr. ${dentistName}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Date</td><td style="padding:6px 0;"><b>${date}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Time</td><td style="padding:6px 0;"><b>${time}</b></td></tr>
    </table>
    <p>See you soon!</p>`);
}

function paymentReceiptEmail({ patientName, totalAmount, paidAmount, status }) {
  return wrapper("Payment Receipt ■", `
    <p>Hi ${patientName},</p>
    <p>Thank you for your payment. Here is a summary:</p>
    <table style="width:100%; border-collapse:collapse; margin:12px 0;">
      <tr><td style="padding:6px 0; color:#64748b;">Total Amount</td><td style="padding:6px 0;"><b>■${totalAmount}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Amount Paid</td><td style="padding:6px 0;"><b>■${paidAmount}</b></td></tr>
      <tr><td style="padding:6px 0; color:#64748b;">Status</td><td style="padding:6px 0;"><b>${status}</b></td></tr>
    </table>
    <p>The detailed receipt is attached as a PDF.</p>`);
}

function medicalReportEmail({ patientName, visitDate }) {
  return wrapper("Your Medical Report ■", `
    <p>Hi ${patientName},</p>
    <p>Your medical report from your visit on <b>${visitDate}</b> is attached as a PDF for your records.</p>`);
}

module.exports = {
  registrationEmail,
  appointmentConfirmationEmail,
  appointmentReminderEmail,
  paymentReceiptEmail,
  medicalReportEmail,
};

```

backend/utls/pdfGenerator.js

WHAT THIS FILE DOES

Generates PDF files entirely on the server using the PDFKit library — this is different from the staff-side invoice PDF, which is generated in the browser with jsPDF. Server-side generation is needed here because these same PDF bytes are also attached directly to outgoing emails.

- Both functions build the PDF into an in-memory buffer instead of writing a file to disk: they listen for data events to collect chunks, and resolve a Promise with `Buffer.concat(chunks)` once PDFKit fires the end event.
- That same Buffer is reused two ways: sent directly as an HTTP response for a browser download, or passed as an email attachment — no need to generate the PDF twice.

THE CODE

```

const PDFDocument = require("pdfkit");

// Both functions return a Promise<Buffer> so callers can either send the PDF
// directly as a download response, or attach the same buffer to an email.

function generateReceiptPDF(invoice, patient) {
  return new Promise((resolve, reject) => {
    const doc = new PDFDocument({ margin: 50 });
    const chunks = [];
    doc.on("data", (c) => chunks.push(c));
    doc.on("end", () => resolve(Buffer.concat(chunks)));
    doc.on("error", reject);

    doc.fontSize(20).fillColor("#0e7490").text("DentalCare Clinic", { align: "center" });
    doc.fontSize(12).fillColor("#64748b").text("Payment Receipt", { align: "center" });
    doc.moveDown(1.5);

    doc.fillColor("#1e293b").fontSize(11);
    doc.text(`Receipt Date: ${new Date(invoice.createdAt).toLocaleDateString()}`);
    doc.text(`Patient: ${patient.name}`);
    doc.text(`Phone: ${patient.phone}`);
    doc.moveDown();

    doc.fontSize(13).fillColor("#0e7490").text("Items", { underline: true });
  });
}

```

```

doc.moveDown(0.3);
doc.fontSize(11).fillColor("#1e293b");
invoice.items.forEach((item) => {
  doc.text(`${item.treatment}`, { continued: true });
  doc.text(`■${item.cost}`, { align: "right" });
});

doc.moveDown();
doc.moveTo(50, doc.y).lineTo(545, doc.y).strokeColor("#cbd5e1").stroke();
doc.moveDown(0.5);

doc.fontSize(11);
doc.text(`Total Amount:`, { continued: true });
doc.text(`■${invoice.totalAmount}`, { align: "right" });
doc.text(`Amount Paid:`, { continued: true });
doc.text(`■${invoice.paidAmount}`, { align: "right" });
doc.text(`Balance Due:`, { continued: true });
doc.text(`■${invoice.totalAmount - invoice.paidAmount}`, { align: "right" });
doc.moveDown();
doc.fontSize(12).fillColor("#0e7490").text(`Status: ${invoice.status}`);

doc.moveDown(2);
doc.fontSize(9).fillColor("#94a3b8").text("Thank you for visiting DentalCare Clinic.", { align: "center" });

doc.end();
});
}

function generateMedicalReportPDF(record, patient) {
  return new Promise((resolve, reject) => {
    const doc = new PDFDocument({ margin: 50 });
    const chunks = [];
    doc.on("data", (c) => chunks.push(c));
    doc.on("end", () => resolve(Buffer.concat(chunks)));
    doc.on("error", reject);

    doc.fontSize(20).fillColor("#0e7490").text("DentalCare Clinic", { align: "center" });
    doc.fontSize(12).fillColor("#64748b").text("Medical Report", { align: "center" });
    doc.moveDown(1.5);

    doc.fillColor("#1e293b").fontSize(11);
    doc.text(`Visit Date: ${new Date(record.visitDate).toLocaleDateString()}`);
    doc.text(`Patient: ${patient.name}`);
    doc.text(`Phone: ${patient.phone}`);
    doc.moveDown();

    const field = (label, value) => {
      doc.fontSize(12).fillColor("#0e7490").text(label);
      doc.fontSize(11).fillColor("#1e293b").text(value || "-");
      doc.moveDown(0.7);
    };

    field("Diagnosis", record.diagnosis);
    field("Treatment Done", record.treatmentDone);
    field("Prescription", record.prescription);
    field("Notes", record.notes);

    doc.moveDown(1.5);
    doc.fontSize(9).fillColor("#94a3b8").text("This report is generated by DentalCare Clinic's management system.", { align: "center" });

    doc.end();
  });
}

module.exports = { generateReceiptPDF, generateMedicalReportPDF };

```

backend/config/cron.js

WHAT THIS FILE DOES

Implements the automatic appointment-reminder feature using the node-cron library.

- `cron.schedule("0 * * * *", ...)` is cron syntax for "run at the top of every hour" — node-cron uses the same 5-field schedule format as Linux's crontab.
- `sendUpcomingReminders()` queries for Scheduled appointments landing in a 24-48 hour window from right now, that haven't already had `reminderSent` set, emails each patient, and marks the ones that succeeded so they're never

emailed twice.

- `startReminderCron()` is called once from `server.js` after the server starts listening, so the schedule only begins after the app is actually up.

THE CODE

```
const cron = require("node-cron");
const Appointment = require("../models/Appointment");
const { sendMail } = require("../mailer");
const { appointmentReminderEmail } = require("../utils/emailTemplates");

// Step 10: Appointment reminder emails.
// Runs every hour, looks for Scheduled appointments happening in the next
// 24-48 hours that haven't had a reminder sent yet, and emails the patient.
async function sendUpcomingReminders() {
  const windowStart = new Date();
  windowStart.setHours(windowStart.getHours() + 24);
  const windowEnd = new Date();
  windowEnd.setHours(windowEnd.getHours() + 48);

  const appointments = await Appointment.find({
    status: "Scheduled",
    reminderSent: { $ne: true },
    date: { $gte: windowStart, $lte: windowEnd },
  })
    .populate("patient", "name email")
    .populate("dentist", "name");

  for (const appt of appointments) {
    if (!appt.patient?.email) continue;
    const sent = await sendMail({
      to: appt.patient.email,
      subject: "Reminder: Your DentalCare Appointment Tomorrow",
      html: appointmentReminderEmail({
        patientName: appt.patient.name,
        dentistName: appt.dentist?.name || "",
        date: new Date(appt.date).toLocaleDateString(),
        time: appt.time,
      }),
    });
    if (sent) {
      appt.reminderSent = true;
      await appt.save();
    }
  }

  if (appointments.length) {
    console.log(`[cron] Processed ${appointments.length} appointment reminder(s)`);
  }
}

function startReminderCron() {
  // Every hour, on the hour. Change the schedule string to run more/less often.
  cron.schedule("0 * * * *", () => {
    sendUpcomingReminders().catch((err) => console.error("[cron] reminder job failed:", err.message));
  });
  console.log("[cron] Appointment reminder job scheduled (runs hourly)");
}

module.exports = { startReminderCron, sendUpcomingReminders };
```

backend/routes/patientAuth.js

WHAT THIS FILE DOES

Registration and login for the patient portal — the patient-facing mirror of `routes/auth.js`, but issuing a token with role: "patient" instead of admin/staff.

- `POST /register` has a nice touch: if the clinic's staff already added this person as a Patient record (with the same email) before they ever created a portal account, registering links the new login to that *existing* record instead of creating a duplicate patient — so their appointment/medical history from before they signed up is already there when they log in.
- On successful registration, it fires off a welcome email via `sendMail()` without awaiting it block the response — the account is created and the token returned immediately either way.

THE CODE

```
const router = require("express").Router();
const jwt = require("jsonwebtoken");
const Patient = require("../models/Patient");
const { sendMail } = require("../config/mailer");
const { registrationEmail } = require("../utils/emailTemplates");

const genToken = (patient) =>
  jwt.sign(
    { id: patient._id, role: "patient", name: patient.name },
    process.env.JWT_SECRET || "change_this_secret_key",
    { expiresIn: "30d" }
  );

// Step 1 & 2: Patient registration + JWT issued immediately on success.
// If a Patient record already exists with this email/phone (added by clinic staff),
// this links portal access to that existing record instead of creating a duplicate.
router.post("/register", async (req, res) => {
  try {
    const { name, email, phone, password } = req.body;
    if (!email || !password) return res.status(400).json({ message: "Email and password are required" });

    let patient = await Patient.findOne({ email });
    if (patient && patient.hasPortalAccess) {
      return res.status(400).json({ message: "An account with this email already exists. Please log in." });
    }

    if (patient) {
      // Existing clinic record (added by staff) – link portal access to it.
      patient.password = password;
      patient.hasPortalAccess = true;
      if (name) patient.name = name;
      if (phone) patient.phone = phone;
    } else {
      patient = new Patient({ name, email, phone, password, hasPortalAccess: true });
    }
    await patient.save();

    // Step 8: Email confirmation on registration (non-blocking – registration
    // still succeeds even if the email fails to send, e.g. SMTP not configured).
    sendMail({ to: email, subject: "Welcome to DentalCare!", html: registrationEmail(patient.name) });

    res.status(201).json({
      token: genToken(patient),
      patient: { id: patient._id, name: patient.name, email: patient.email },
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// Step 1 & 2: Patient login
router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;
    const patient = await Patient.findOne({ email, hasPortalAccess: true }).select("+password");
    if (!patient) return res.status(400).json({ message: "Invalid credentials" });
    const match = await patient.comparePassword(password);
    if (!match) return res.status(400).json({ message: "Invalid credentials" });

    res.json({
      token: genToken(patient),
      patient: { id: patient._id, name: patient.name, email: patient.email },
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

module.exports = router;
```

backend/routes/portal.js

WHAT THIS FILE DOES

The heart of the patient portal — every route here is scoped to *only* the logged-in patient's own data, using `req.user.id` (taken from their JWT) as the filter in every database query. A patient can never see another patient's records, even if they guessed a valid MongoDB id.

- GET /dashboard fetches the patient plus all their appointments/invoices/records in parallel, then calculates derived numbers in JavaScript (next upcoming appointment, completed-visit count, outstanding balance) rather than storing them — always freshly correct.
- GET /invoices/:id/pdf and GET /records/:id/pdf call the PDF generator and stream the resulting buffer straight back as the response body with Content-Disposition: attachment, which makes the browser download it instead of trying to display it inline.
- POST /invoices/:id/email and POST /records/:id/email are the 'Email me' buttons on the frontend — they regenerate the same PDF and send it through sendMail() on demand, returning whether it actually sent so the frontend can show an honest message even if SMTP isn't configured.

THE CODE

```
const router = require("express").Router();
const Patient = require("../models/Patient");
const Appointment = require("../models/Appointment");
const Invoice = require("../models/Invoice");
const MedicalRecord = require("../models/MedicalRecord");
const auth = require("../middleware/auth");
const patientOnly = require("../middleware/patientOnly");
const { sendMail } = require("../config/mailer");
const { paymentReceiptEmail, medicalReportEmail } = require("../utils/emailTemplates");
const { generateReceiptPDF, generateMedicalReportPDF } = require("../utils/pdfGenerator");

// Every route below requires a valid PATIENT portal login (not a staff token).
router.use(auth, patientOnly);

// Step 3: Patient's own dashboard summary
router.get("/dashboard", async (req, res) => {
  const patientId = req.user.id;
  const [patient, appointments, invoices, records] = await Promise.all([
    Patient.findById(patientId),
    Appointment.find({ patient: patientId }).populate("dentist", "name specialization").sort({ date: 1 }),
    Invoice.find({ patient: patientId }),
    MedicalRecord.find({ patient: patientId }),
  ]);

  const now = new Date();
  const nextAppointment = appointments.find((a) => a.status === "Scheduled" && new Date(a.date) >= now) || null;
  const totalVisits = appointments.filter((a) => a.status === "Completed").length;
  const outstandingBalance = invoices.reduce((sum, inv) => sum + (inv.totalAmount - inv.paidAmount), 0);

  res.json({
    patient: { name: patient.name, email: patient.email, phone: patient.phone },
    nextAppointment,
    totalVisits,
    totalAppointments: appointments.length,
    outstandingBalance,
    totalRecords: records.length,
  });
});

// Step 4: Appointment history
router.get("/appointments", async (req, res) => {
  const appointments = await Appointment.find({ patient: req.user.id })
    .populate("dentist", "name specialization")
    .sort({ date: -1 });
  res.json(appointments);
});

// Step 5: Payment history
router.get("/invoices", async (req, res) => {
  const invoices = await Invoice.find({ patient: req.user.id }).sort({ createdAt: -1 });
  res.json(invoices);
});

// Step 6: Medical reports
router.get("/records", async (req, res) => {
  const records = await MedicalRecord.find({ patient: req.user.id })
    .populate("dentist", "name specialization")
    .sort({ visitDate: -1 });
  res.json(records);
});

// Step 7: PDF download – payment receipt
router.get("/invoices/:id/pdf", async (req, res) => {
  const invoice = await Invoice.findOne({ _id: req.params.id, patient: req.user.id });
  if (!invoice) return res.status(404).json({ message: "Invoice not found" });
  const patient = await Patient.findById(req.user.id);
```

```

const pdfBuffer = await generateReceiptPDF(invoice, patient);
res.set({ "Content-Type": "application/pdf", "Content-Disposition": `attachment; filename=receipt-${invoice._id}.pdf` });
res.send(pdfBuffer);
});

// Step 7: PDF download – medical report
router.get("/records/:id/pdf", async (req, res) => {
  const record = await MedicalRecord.findOne({ _id: req.params.id, patient: req.user.id });
  if (!record) return res.status(404).json({ message: "Record not found" });
  const patient = await Patient.findById(req.user.id);
  const pdfBuffer = await generateMedicalReportPDF(record, patient);
  res.set({ "Content-Type": "application/pdf", "Content-Disposition": `attachment; filename=medical-report-${record._id}.pdf` });
  res.send(pdfBuffer);
});

// Step 9: Re-send payment receipt by email on demand
router.post("/invoices/:id/email", async (req, res) => {
  const invoice = await Invoice.findOne({ _id: req.params.id, patient: req.user.id });
  if (!invoice) return res.status(404).json({ message: "Invoice not found" });
  const patient = await Patient.findById(req.user.id);
  if (!patient.email) return res.status(400).json({ message: "No email on file" });

  const pdfBuffer = await generateReceiptPDF(invoice, patient);
  const sent = await sendMail({
    to: patient.email,
    subject: "Your DentalCare Payment Receipt",
    html: paymentReceiptEmail({ patientName: patient.name, totalAmount: invoice.totalAmount, paidAmount: invoice.paidAmount,
    status: invoice.status }),
    attachments: [{ filename: "receipt.pdf", content: pdfBuffer }],
  });
  res.json({ sent, message: sent ? "Receipt emailed" : "Email not sent – SMTP not configured on the server" });
});

// Step 9: Email a medical report on demand
router.post("/records/:id/email", async (req, res) => {
  const record = await MedicalRecord.findOne({ _id: req.params.id, patient: req.user.id });
  if (!record) return res.status(404).json({ message: "Record not found" });
  const patient = await Patient.findById(req.user.id);
  if (!patient.email) return res.status(400).json({ message: "No email on file" });

  const pdfBuffer = await generateMedicalReportPDF(record, patient);
  const sent = await sendMail({
    to: patient.email,
    subject: "Your DentalCare Medical Report",
    html: medicalReportEmail({ patientName: patient.name, visitDate: new Date(record.visitDate).toLocaleDateString() }),
    attachments: [{ filename: "medical-report.pdf", content: pdfBuffer }],
  });
  res.json({ sent, message: sent ? "Report emailed" : "Email not sent – SMTP not configured on the server" });
});

module.exports = router;

```

Emails wired into existing staff routes

Two of the existing staff-side routes were extended to trigger patient emails automatically, so staff don't have to do anything extra — the emails just happen as a side effect of normal clinic work.

- `backend/routes/appointments.js` — after POST / creates the appointment, if the patient has an email on file, it sends the booking confirmation email.
- `backend/routes/invoices.js` — after POST / creates the invoice, it generates the receipt PDF and emails it to the patient in the same request.

Frontend additions

frontend/src/api/patientAxios.js

WHAT THIS FILE DOES

A second Axios instance, separate from the staff one. It reads its token from a different localStorage key (`patientToken` instead of `token`), which is what allows a staff session and a patient session to coexist in the same browser without overwriting each other.

THE CODE

```
import axios from "axios";

// A separate Axios instance for the patient portal, so a patient's login
// session never collides with a staff/admin session in the same browser
// (each uses its own localStorage key and its own token).
const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:5000/api";

const patientApi = axios.create({ baseURL: API_BASE });

patientApi.interceptors.request.use((config) => {
  const token = localStorage.getItem("patientToken");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default patientApi;
```

frontend/src/context/PatientAuthContext.jsx

WHAT THIS FILE DOES

The patient-portal mirror of AuthContext.jsx — same shape (login/register/logout, persisted to localStorage) but talking to /patient-auth/... endpoints via patientAxios and storing under the patientUser key.

THE CODE

```
import { createContext, useContext, useState } from "react";
import patientApi from "../api/patientAxios";

const PatientAuthContext = createContext(null);

export function PatientAuthProvider({ children }) {
  const [patient, setPatient] = useState(() => {
    const stored = localStorage.getItem("patientUser");
    return stored ? JSON.parse(stored) : null;
  });

  const login = async (email, password) => {
    const { data } = await patientApi.post("/patient-auth/login", { email, password });
    localStorage.setItem("patientToken", data.token);
    localStorage.setItem("patientUser", JSON.stringify(data.patient));
    setPatient(data.patient);
  };

  const register = async (form) => {
    const { data } = await patientApi.post("/patient-auth/register", form);
    localStorage.setItem("patientToken", data.token);
    localStorage.setItem("patientUser", JSON.stringify(data.patient));
    setPatient(data.patient);
  };

  const logout = () => {
    localStorage.removeItem("patientToken");
    localStorage.removeItem("patientUser");
    setPatient(null);
  };

  return (
    <PatientAuthContext.Provider value={{ patient, login, register, logout }}>
      {children}
    </PatientAuthContext.Provider>
  );
}

export const usePatientAuth = () => useContext(PatientAuthContext);
```

frontend/src/components/PatientPrivateRoute.jsx

WHAT THIS FILE DOES

The patient-portal mirror of PrivateRoute.jsx: redirects to /patient-login instead of /login if there's no logged-in patient in context.

THE CODE

```
import { Navigate } from "react-router-dom";
import { usePatientAuth } from "../context/PatientAuthContext";

export default function PatientPrivateRoute({ children }) {
  const { patient } = usePatientAuth();
  if (!patient) return <Navigate to="/patient-login" replace />;
  return children;
}
```

frontend/src/components/PatientLayout.jsx

WHAT THIS FILE DOES

Same idea as the staff Layout.jsx, but pairs page content with <PatientSidebar /> instead of the staff Sidebar.

THE CODE

```
import PatientSidebar from "../PatientSidebar";

export default function PatientLayout({ children }) {
  return (
    <div className="flex min-h-screen bg-slate-50 dark:bg-slate-900">
      <PatientSidebar />
      <main className="flex-1 p-8">{children}</main>
    </div>
  );
}
```

frontend/src/components/PatientSidebar.jsx

WHAT THIS FILE DOES

A simpler navigation menu than the staff Sidebar — only 4 links (Dashboard, My Appointments, My Payments, Medical Reports) since patients don't need staff-only sections like Staff Management or the service price list. Reuses the same ThemeToggle component from the staff app.

THE CODE

```
import { NavLink, useNavigate } from "react-router-dom";
import { usePatientAuth } from "../context/PatientAuthContext";
import ThemeToggle from "../ThemeToggle";

const links = [
  { to: "/patient/dashboard", label: "Dashboard", icon: "■" },
  { to: "/patient/appointments", label: "My Appointments", icon: "■" },
  { to: "/patient/payments", label: "My Payments", icon: "■" },
  { to: "/patient/records", label: "Medical Reports", icon: "■" },
];

export default function PatientSidebar() {
  const { patient, logout } = usePatientAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/patient-login");
  };

  return (
    <aside className="w-64 bg-slate-900 text-white min-h-screen flex flex-col">
      <div className="p-5 border-b border-slate-700">
        <h1 className="text-xl font-bold text-cyan-400">■ DentalCare</h1>
        <p className="text-xs text-slate-400 mt-1">Patient Portal</p>
      </div>
      <nav className="flex-1 p-3 space-y-1">
        {links.map((link) => (
          <NavLink
            key={link.to}
            to={link.to}
            className={({ isActive }) =>
              `flex items-center gap-3 px-4 py-2.5 rounded-lg text-sm transition ${
                isActive ? "bg-cyan-600 text-white" : "text-slate-300 hover:bg-slate-800"
              }`
          >
            <span>{link.icon}</span>
            {link.label}
          </NavLink>
        ))}
      </nav>
    </aside>
  );
}
```

```

    }`
  }
  >
  <span>{link.icon}</span> {link.label}
</NavLink>
)))}
</nav>
<div className="p-4 border-t border-slate-700 space-y-3">
  <ThemeToggle />
  <p className="text-sm text-slate-300">{patient?.name}</p>
  <p className="text-xs text-slate-500 truncate">{patient?.email}</p>
  <button
    onClick={handleLogout}
    className="w-full bg-slate-800 hover:bg-red-600 text-sm py-2 rounded-lg transition"
  >
    Logout
  </button>
</div>
</aside>
);
}

```

frontend/src/pages/patient/PatientLogin.jsx

WHAT THIS FILE DOES

Nearly identical in structure to the staff Login.jsx, but calls usePatientAuth() instead of useAuth() and redirects to /patient/dashboard on success. Includes a link back to the staff login page for anyone who lands here by mistake.

THE CODE

```

import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { usePatientAuth } from "../../../context/PatientAuthContext";

export default function PatientLogin() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { login } = usePatientAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await login(email, password);
      navigate("/patient/dashboard");
    } catch (err) {
      setError(err.response?.data?.message || "Login failed");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-100">
      <form onSubmit={handleSubmit} className="bg-white p-8 rounded-xl shadow-md w-full max-w-sm">
        <h1 className="text-2xl font-bold text-center text-slate-800 mb-1">■ DentalCare</h1>
        <p className="text-center text-slate-500 text-sm mb-6">Patient Portal Login</p>
        {error && <p className="bg-red-50 text-red-600 text-sm p-2 rounded mb-4">{error}</p>}
        <label className="text-sm text-slate-600">Email</label>
        <input
          type="email"
          required
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500"
        />
        <label className="text-sm text-slate-600">Password</label>
        <input
          type="password"
          required
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="w-full border rounded-lg p-2.5 mt-1 mb-6 outline-cyan-500"

```

```

    />
    <button disabled={loading} className="w-full bg-cyan-600 hover:bg-cyan-700 disabled:opacity-60 text-white py-2.5
rounded-lg font-medium transition">
      {loading ? "Logging in..." : "Login"}
    </button>
    <p className="text-center text-sm text-slate-500 mt-4">
      New patient? <Link to="/patient-register" className="text-cyan-600 font-medium">Create an account</Link>
    </p>
    <p className="text-center text-xs text-slate-400 mt-4">
      Clinic staff? <Link to="/login" className="text-cyan-600">Staff login here</Link>
    </p>
  </form>
</div>
);
}

```

frontend/src/pages/patient/PatientRegister.jsx

WHAT THIS FILE DOES

Collects name, email, phone, and password. A short note under the form tells returning patients to use the same email/phone the clinic already has on file, so the backend's existing-record-linking logic (described above in patientAuth.js) connects their new login to their real history.

THE CODE

```

import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { usePatientAuth } from "../../context/PatientAuthContext";

export default function PatientRegister() {
  const [form, setForm] = useState({ name: "", email: "", phone: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { register } = usePatientAuth();
  const navigate = useNavigate();

  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await register(form);
      navigate("/patient/dashboard");
    } catch (err) {
      setError(err.response?.data?.message || "Registration failed");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-100">
      <form onSubmit={handleSubmit} className="bg-white p-8 rounded-xl shadow-md w-full max-w-sm">
        <h1 className="text-2xl font-bold text-center text-slate-800 mb-1">■ DentalCare</h1>
        <p className="text-center text-slate-500 text-sm mb-6">Create Your Patient Account</p>
        {error && <p className="bg-red-50 text-red-600 text-sm p-2 rounded mb-4">{error}</p>}
        <label className="text-sm text-slate-600">Full Name</label>
        <input name="name" required value={form.name} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500" />
        <label className="text-sm text-slate-600">Email</label>
        <input type="email" name="email" required value={form.email} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500" />
        <label className="text-sm text-slate-600">Phone</label>
        <input name="phone" required value={form.phone} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-4 outline-cyan-500" />
        <label className="text-sm text-slate-600">Password</label>
        <input type="password" name="password" required value={form.password} onChange={handleChange} className="w-full border rounded-lg p-2.5 mt-1 mb-6 outline-cyan-500" />
        <button disabled={loading} className="w-full bg-cyan-600 hover:bg-cyan-700 disabled:opacity-60 text-white py-2.5 rounded-lg font-medium transition">
          {loading ? "Creating account..." : "Create Account"}
        </button>
        <p className="text-center text-sm text-slate-500 mt-4">
          Already have an account? <Link to="/patient-login" className="text-cyan-600 font-medium">Login</Link>
        </p>
        <p className="text-center text-xs text-slate-400 mt-3">

```

```

        If the clinic already has your records on file, use the same email/phone
        they have &mdash; your existing history will be linked automatically.
    </p>
</form>
</div>
);
}

```

frontend/src/pages/patient/PatientDashboard.jsx

WHAT THIS FILE DOES

The patient's home screen: fetches `/portal/dashboard` once on load and renders 4 stat cards plus a "Next Appointment" card. Also includes a small info banner reminding the patient that confirmations, reminders, and receipts are sent automatically by email.

THE CODE

```

import { useEffect, useState } from "react";
import PatientLayout from "../../components/PatientLayout";
import StatCard from "../../components/StatCard";
import patientApi from "../../api/patientAxios";

export default function PatientDashboard() {
  const [data, setData] = useState(null);
  const [error, setError] = useState("");

  useEffect(() => {
    patientApi
      .get("/portal/dashboard")
      .then((res) => setData(res.data))
      .catch(() => setError("Could not load your dashboard. Please try logging in again."));
  }, []);

  return (
    <PatientLayout>
      <h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">
        Welcome back{data ? ``, `${data.patient.name}` : ""} █
      </h1>
      <p className="text-slate-500 dark:text-slate-400 mb-6">Here's a summary of your dental care</p>

      {error && <p className="bg-red-50 text-red-600 p-3 rounded-lg mb-6 text-sm">{error}</p>}

      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <StatCard label="Total Appointments" value={data?.totalAppointments ?? "-"} icon="█" color="bg-cyan-50" />
        <StatCard label="Completed Visits" value={data?.totalVisits ?? "-"} icon="█" color="bg-green-50" />
        <StatCard label="Medical Reports" value={data?.totalRecords ?? "-"} icon="█" color="bg-purple-50" />
        <StatCard label="Outstanding Balance" value={data ? `█${data.outstandingBalance}` : "-"} icon="█" color="bg-amber-50" />
      </div>

      <div className="mt-8 bg-white dark:bg-slate-800 rounded-xl shadow-sm p-6 border border-slate-100 dark:border-slate-700">
        <h2 className="font-semibold text-slate-800 dark:text-white mb-3">Next Appointment</h2>
        {data?.nextAppointment ? (
          <div className="text-slate-700 dark:text-slate-200">
            <p className="text-lg font-semibold">
              {new Date(data.nextAppointment.date).toLocaleDateString()} at {data.nextAppointment.time}
            </p>
            <p className="text-sm text-cyan-600 mt-1">
              Dr. {data.nextAppointment.dentist?.name} &mdash; {data.nextAppointment.dentist?.specialization}
            </p>
            {data.nextAppointment.reason && (
              <p className="text-sm text-slate-500 dark:text-slate-400 mt-1">Reason: {data.nextAppointment.reason}</p>
            )}
          </div>
        ) : (
          <p className="text-slate-400">No upcoming appointments scheduled.</p>
        )}
      </div>

      <div className="mt-4 bg-cyan-50 dark:bg-slate-800 border border-cyan-100 dark:border-slate-700 rounded-xl p-4 text-sm text-cyan-800 dark:text-cyan-300">
        █ You'll receive an email confirmation whenever an appointment is booked, plus a reminder the day
        before, and a receipt whenever a payment is recorded.
      </div>
    </PatientLayout>
  );
}

```

frontend/src/pages/patient/PatientAppointments.jsx

WHAT THIS FILE DOES

A read-only table of the patient's own appointment history (Step 4) — deliberately simpler than the staff Appointments page since patients can only view, not book, cancel, or edit here.

THE CODE

```
import { useEffect, useState } from "react";
import PatientLayout from "../../../components/PatientLayout";
import patientApi from "../../../api/patientAxios";

export default function PatientAppointments() {
  const [appointments, setAppointments] = useState([]);

  useEffect(() => {
    patientApi.get("/portal/appointments").then((res) => setAppointments(res.data));
  }, []);

  const statusColor = {
    Scheduled: "bg-amber-100 text-amber-700",
    Completed: "bg-green-100 text-green-700",
    Cancelled: "bg-red-100 text-red-700",
  };

  return (
    <PatientLayout>
      <h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">My Appointments</h1>
      <p className="text-slate-500 dark:text-slate-400 mb-6">Your full appointment history</p>

      <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 overflow-x-auto">
        <table className="w-full text-sm">
          <thead className="bg-slate-50 dark:bg-slate-900 text-slate-500 dark:text-slate-400 text-left">
            <tr>
              <th className="p-3">Date</th>
              <th className="p-3">Time</th>
              <th className="p-3">Dentist</th>
              <th className="p-3">Reason</th>
              <th className="p-3">Status</th>
            </tr>
          </thead>
          <tbody>
            {appointments.map((a) => (
              <tr key={a._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
                <td className="p-3">{new Date(a.date).toLocaleDateString()}</td>
                <td className="p-3">{a.time}</td>
                <td className="p-3">Dr. {a.dentist?.name} <span className="text-xs text-slate-400">({a.dentist?.specialization})</span></td>
                <td className="p-3">{a.reason || "-"}</td>
                <td className="p-3">
                  <span className={`px-2 py-1 rounded-full text-xs font-medium ${statusColor[a.status]}`}>{a.status}</span>
                </td>
              </tr>
            ))}
            {appointments.length === 0 && (
              <tr><td colspan="5" className="p-6 text-center text-slate-400">No appointments yet</td></tr>
            )}
          </tbody>
        </table>
      </div>
    </PatientLayout>
  );
}
```

frontend/src/pages/patient/PatientPayments.jsx

WHAT THIS FILE DOES

Payment history (Step 5) plus two actions per invoice: `downloadPdf()` requests the PDF as a blob and triggers a browser download the same way the staff CSV export does (Step 7), and `emailReceipt()` calls the on-demand email endpoint and shows the server's response message in a toast (Step 9) — including an honest 'SMTP not configured' message if email isn't set up.

THE CODE

```
import { useEffect, useState } from "react";
import PatientLayout from "../../components/PatientLayout";
import patientApi from "../../api/patientAxios";
import { useToast } from "../../context/ToastContext";

export default function PatientPayments() {
  const [invoices, setInvoices] = useState([]);
  const [busyId, setBusyId] = useState(null);
  const { showToast } = useToast();

  useEffect(() => {
    patientApi.get("/portal/invoices").then((res) => setInvoices(res.data));
  }, []);

  const statusColor = {
    Paid: "bg-green-100 text-green-700",
    "Partially Paid": "bg-amber-100 text-amber-700",
    Unpaid: "bg-red-100 text-red-700",
  };

  // Step 7: PDF download
  const downloadPdf = async (id) => {
    setBusyId(id);
    try {
      const res = await patientApi.get(`/portal/invoices/${id}/pdf`, { responseType: "blob" });
      const url = window.URL.createObjectURL(new Blob([res.data], { type: "application/pdf" }));
      const link = document.createElement("a");
      link.href = url;
      link.download = `receipt-${id}.pdf`;
      link.click();
      window.URL.revokeObjectURL(url);
    } catch {
      showToast("Could not download receipt", "error");
    } finally {
      setBusyId(null);
    }
  };

  // Step 9: Email the receipt again
  const emailReceipt = async (id) => {
    setBusyId(id);
    try {
      const res = await patientApi.post(`/portal/invoices/${id}/email`);
      showToast(res.data.message, res.data.sent ? "success" : "info");
    } catch {
      showToast("Could not email receipt", "error");
    } finally {
      setBusyId(null);
    }
  };

  return (
    <PatientLayout>
      <h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">My Payments</h1>
      <p className="text-slate-500 dark:text-slate-400 mb-6">Your billing and payment history</p>

      <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100 dark:border-slate-700 overflow-x-auto">
        <table className="w-full text-sm">
          <thead className="bg-slate-50 dark:bg-slate-900 text-slate-500 dark:text-slate-400 text-left">
            <tr>
              <th className="p-3">Date</th>
              <th className="p-3">Total</th>
              <th className="p-3">Paid</th>
              <th className="p-3">Balance</th>
              <th className="p-3">Status</th>
              <th className="p-3">Actions</th>
            </tr>
          </thead>
          <tbody>
            {invoices.map((inv) => (
              <tr key={inv._id} className="border-t border-slate-100 dark:border-slate-700 dark:text-slate-200">
                <td className="p-3">{new Date(inv.createdAt).toLocaleDateString()}</td>
                <td className="p-3">■{inv.totalAmount}</td>
                <td className="p-3">■{inv.paidAmount}</td>
                <td className="p-3">■{inv.totalAmount - inv.paidAmount}</td>
                <td className="p-3"><span className={`px-2 py-1 rounded-full text-xs font-medium ${statusColor[inv.status]}`}>{inv.status}</span></td>
                <td className="p-3 space-x-2">
                  <button disabled={busyId === inv._id} onClick={() => downloadPdf(inv._id)} className="text-cyan-600 hover:underline disabled:opacity-50">
                    Download PDF
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </PatientLayout>
  );
}
```

```

        </button>
        <button disabled={busyId === inv._id} onClick={() => emailReceipt(inv._id)} className="text-slate-500
hover:underline disabled:opacity-50">
            Email me
        </button>
    </td>
</tr>
</tbody>
</table>
</div>
</PatientLayout>
);
}

```

frontend/src/pages/patient/PatientRecords.jsx

WHAT THIS FILE DOES

Medical report history (Step 6), structured almost identically to PatientPayments.jsx: the same download-PDF and email-me pattern, just pointed at the records endpoints instead of the invoice ones (Steps 7 & 9).

THE CODE

```

import { useEffect, useState } from "react";
import PatientLayout from "../../components/PatientLayout";
import patientApi from "../../api/patientAxios";
import { useToast } from "../../context/ToastContext";

export default function PatientRecords() {
    const [records, setRecords] = useState([]);
    const [busyId, setBusyId] = useState(null);
    const { showToast } = useToast();

    useEffect(() => {
        patientApi.get("/portal/records").then((res) => setRecords(res.data));
    }, []);

    // Step 7: PDF download
    const downloadPdf = async (id) => {
        setBusyId(id);
        try {
            const res = await patientApi.get(`/portal/records/${id}/pdf`, { responseType: "blob" });
            const url = window.URL.createObjectURL(new Blob([res.data], { type: "application/pdf" }));
            const link = document.createElement("a");
            link.href = url;
            link.download = `medical-report-${id}.pdf`;
            link.click();
            window.URL.revokeObjectURL(url);
        } catch {
            showToast("Could not download report", "error");
        } finally {
            setBusyId(null);
        }
    };

    // Step 9: Email the report
    const emailReport = async (id) => {
        setBusyId(id);
        try {
            const res = await patientApi.post(`/portal/records/${id}/email`);
            showToast(res.data.message, res.data.sent ? "success" : "info");
        } catch {
            showToast("Could not email report", "error");
        } finally {
            setBusyId(null);
        }
    };

    return (
        <PatientLayout>
            <h1 className="text-2xl font-bold text-slate-800 dark:text-white mb-1">My Medical Reports</h1>
            <p className="text-slate-500 dark:text-slate-400 mb-6">Diagnosis, treatment, and prescription history</p>

            <div className="space-y-3">
                {records.length === 0 && (

```

```

        <p className="text-slate-400 text-center py-8 bg-white dark:bg-slate-800 rounded-xl border border-slate-100
dark:border-slate-700">
            No medical reports on file yet.
        </p>
    )}
    {records.map((r) => (
        <div key={r._id} className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-slate-100
dark:border-slate-700 p-5">
            <div className="flex justify-between items-start mb-2">
                <div>
                    <p className="font-semibold text-slate-800 dark:text-white">{new Date(r.visitDate).toLocaleDateString()}</p>
                    {r.dentist?.name && <p className="text-xs text-cyan-600">Dr. {r.dentist.name} &bull;
{r.dentist.specialization}</p>}
                </div>
                <div className="space-x-3 text-sm">
                    <button disabled={busyId === r._id} onClick={() => downloadPdf(r._id)} className="text-cyan-600 hover:underline
disabled:opacity-50">
                        Download PDF
                    </button>
                    <button disabled={busyId === r._id} onClick={() => emailReport(r._id)} className="text-slate-500 hover:underline
disabled:opacity-50">
                        Email me
                    </button>
                </div>
            </div>
            <div className="grid grid-cols-1 md:grid-cols-2 gap-3 text-sm">
                {r.diagnosis && <div><span className="text-slate-400">Diagnosis</span><p className="text-slate-700
dark:text-slate-200">{r.diagnosis}</p></div>}
                {r.treatmentDone && <div><span className="text-slate-400">Treatment Done</span><p className="text-slate-700
dark:text-slate-200">{r.treatmentDone}</p></div>}
                {r.prescription && <div className="md:col-span-2"><span className="text-slate-400">Prescription</span><p
className="text-slate-700 dark:text-slate-200">{r.prescription}</p></div>}
                {r.notes && <div className="md:col-span-2"><span className="text-slate-400">Notes</span><p
className="text-slate-700 dark:text-slate-200">{r.notes}</p></div>}
            </div>
        </div>
    )})
    </div>
</PatientLayout>
    );
}

```

How the 10 steps map to files

- **Step 1-2: Registration/Login + JWT** — backend/routes/patientAuth.js, frontend PatientAuthContext.jsx
- **Step 3: User dashboard** — backend/routes/portal.js (GET /dashboard), PatientDashboard.jsx
- **Step 4: Appointment history** — backend/routes/portal.js (GET /appointments), PatientAppointments.jsx
- **Step 5: Payment history** — backend/routes/portal.js (GET /invoices), PatientPayments.jsx
- **Step 6: Medical reports** — backend/routes/portal.js (GET /records), PatientRecords.jsx
- **Step 7: PDF download** — backend/utlis/pdfGenerator.js + the two /pdf routes in portal.js
- **Step 8: Email confirmation** — backend/routes/appointments.js (POST /) + patientAuth.js (register)
- **Step 9: Email report + receipt** — backend/routes/invoices.js (POST /) + the two /email routes in portal.js
- **Step 10: Appointment reminder emails** — backend/config/cron.js

Explaining It in One Breath

If a friend asks "how does this whole thing work?", the short version is: **React pages call the Axios instance → Axios attaches your login token automatically → the request hits an Express route → the auth middleware checks the token → the route talks to a Mongoose model → MongoDB stores or returns the data → Express sends JSON back → React updates the screen.** Every single feature in this app, from logging in to downloading a PDF invoice, is just that loop happening with a different route and a different page. The patient portal follows the exact same loop — it just uses a second token, a second Axios instance, and routes scoped to one patient's own data — plus one extra step at the end where certain actions (booking, paying, reminders) also trigger an email as a side effect.